

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО”
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

КУРСОВА РОБОТА
з дисципліни «Інженерія програмного забезпечення»
на тему: “Тетріс”

Студентки II курсу групи ІО-32
напряму підготовки: 123 – Комп’ютерна інженерія
номер залікової № ІО-3213
Куліченко Софії Русланівни

Керівник:

Асистент

Русінов Володимир Володимирович

Київ – 2025 р.

Зміст

ВСТУП.....	4
1. Технічне Завдання.....	5
1.1. Загальне Завдання	5
1.2. Можливості програми	5
2. Планування	5
2.1. Розподілення роботи	5
2.2. Планування робіт	6
3. Підготовка інфраструктури	15
3.1. Допоміжні забезпечення.....	15
3.2. Інфраструктура - Jira	16
3.3. Часові витрати – Clockify	18
3.4. Контроль версій – Git	20
4. Гра.....	21
4.1. Проектування гри	21
4.2. Візуальна частина гри	22
4.3. Безкінечний ігровий режим	28
4.4. Рівневий ігровий режим	28
4.5. Код до гри	29
4.6. Звуковий супровід.....	29
4.7. Інфраструктура - «Atlassian Jira».....	29
5. База даних.....	30
5.1. Проектування бази даних.....	30
5.2. Технічна підготовка віддаленого підключення	31
6. Сайт.....	35
6.1. Проектування сайту.....	35
6.2. Реалізація бази даних – рівень data	37
6.3. SQL запити – рівень service	40
6.4. HTTP запити – рівень web	43
6.5. Фронтенд	45
6.6. Передача даних з гри.....	45
6.7. Взаємодія гри із сайтом	45
7. Тестування	45
7.1. Модульне тестування	45
7.1.1 Тестування підключення до БД.....	46
7.1.2 Тестування розробки гри на першому етапі	46
7.1.3 Тестування розробки гри на другому етапі.....	47
7.1.4 Тестування розробки гри на третьому етапі	48

7.1.5 Тестування HTTP запитів	49
7.2 Тестування готового проекту для перевірки працездатності системи	50
Висновок	59
Джерела	60
Додаток А	61
Додаток Б	69

ВСТУП

Метою даної курсової роботи є розробка гри за тематикою «Тетріс», яка має доповнення до основного набору механік. А також публікація гри на сайті, який має підключення до віддаленої бази даних.

В ході розробки гри було виконано декілька типів ігрових блоків, два ігрові режими(рівневий та безкінечний), а також розроблено інтуїтивно зрозумілий графічний інтерфейс.

В ході розробки сайту реалізовано отримання запитів від користувачів та подальша передача даних на бекенд з фіксацією в базі даних.

Виконаний проект є готовою версією сайту для розміщення на локальному або хмарному хостингу.

1. Технічне Завдання

1.1. Загальне Завдання

Завданням курсової роботи є реалізація гри за допомогою ігрового рушія «Unity» та подальша її публікація за допомогою хостингу на сайті створеного на фреймворці .NET core 9 та Vue+Vite. А також створення та менеджмент бази даних PostgreSQL на одноплатному комп'ютері Raspberry Pi 3b.

1.2. Можливості програми

Розроблена програма повинна мати наступні функціональні можливості:

- реалізація ігрової механіки «Тетріс»
- доступ та використання гри через веб-браузер
- авторизація користувачів
- система обліку рахунку
- ранжування користувачів за рахунком
- реалізація двох режимів гри: з рівнями та безкінечний
- захист пароллю користувача, незворотне криптоперетворення
- керування обліковим записом користувача
- система допомоги користувачу: контекстозалежні підказки, інструкція управління
 - звуковий супровід

Не функціональні можливості програми:

- можливість розподілення елементів програми на різних серверах, а саме: фронтенд, бекенд, база даних
- розмежування доступу до елементів програми
- забезпечення резервного копіювання елементів програми без зупинки сервісу

2. Планування

2.1. Розподілення роботи

При створенні проекту було задіяно дві особи. Планування та менеджмент робіт виконувались з використанням інструментів наданих хмарним рішенням «Atlassian | Jira». Облік часу виконання робіт для відслідковування навантаження робіт виконувались з використанням інструментів наданих хмарним рішенням «Clockify».

При розподілення робіт по виконавцям була використана та погоджена наступна матриця відповідальності.(див. табл. 2.1.1)

Таблиця 2.1.1 – Матриця відповідальності

Номер	Сфера відповідальності	ПІБ
1	Планування робіт	Куліченко С.Р. [90%]
2		Оніщенко Є.О. [10%]
3	Налаштування проекту та користувачів в «Atlassian Jira»	Куліченко С.Р.
4	Налаштування проекту та користувачів в «Github»	Куліченко С.Р.
5	Налаштування проекту та користувачів в «Clockify»	Куліченко С.Р.
6	Проектування проекту	Куліченко С.Р.
7	Розробка візуального аспекту гри	Куліченко С.Р.
8	Проектування ігрової механіки	Куліченко С.Р. [60%]
9		Оніщенко Є.О. [40%]
10	Реалізація ігрової механіки	Оніщенко Є.О.
11	Проектування баз даних	Куліченко С.Р.
12	Реалізація баз даних	Куліченко С.Р. [60%]
13		Оніщенко Є.О. [40%]
14	Проектування веб-сайту	Куліченко С.Р.
15	Передача даних з гри	Оніщенко Є.О.
16	Реалізація http запитів	Куліченко С.Р. [80%]
17		Оніщенко Є.О. [20%]
18	Розробка фронтенд	Куліченко С.Р. [20%]
19		Оніщенко Є.О. [80%]
20	Забезпечення авторизації користувача	Куліченко С.Р. [50%]
21		Оніщенко Є.О. [50%]
22	Реалізація запитів на базу даних	Куліченко С.Р.
23	Додання звукового супроводу	Оніщенко Є.О.
24	Реалізація віддаленого підключення БД	Куліченко С.Р.

2.2. Планування робіт

Для планування робіт був відкритий проект «Atlassian | Jira».(див.рис. 2.2.1)

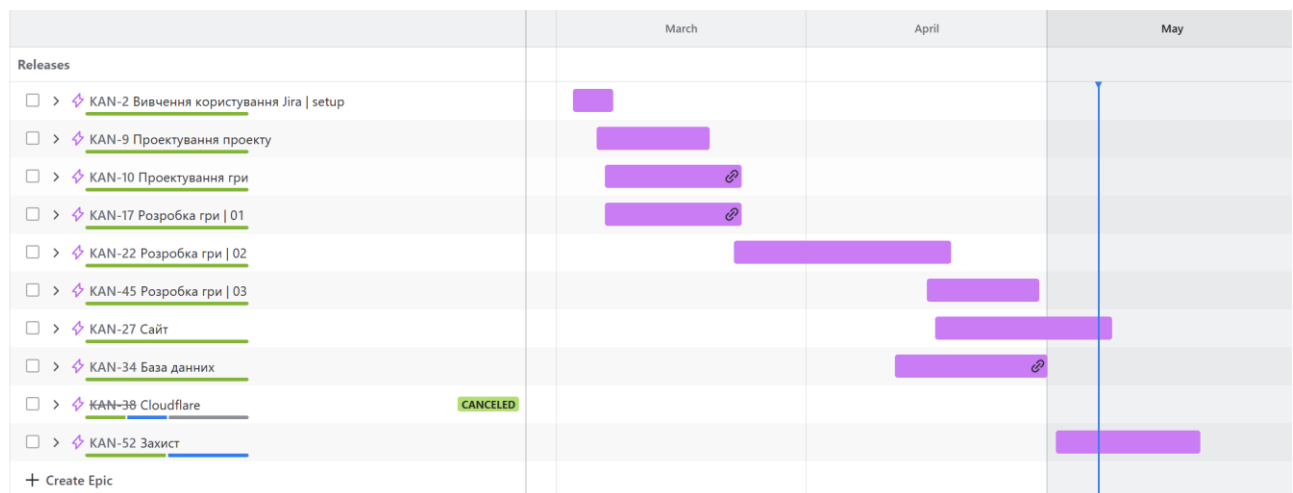


Рисунок 2.2.1 – проект «Jira»

Задачі були зібрані в епіки, які мали свої часові відрізки розраховані на виконання всіх задач вмісту. Розглянемо більш детально наповнення епіків.

Kan-2 покриває собою все налаштування проекту «Atlassian | Jira» та проекту «Clockify» перед початком роботи. (див. рис. 2.2.2)

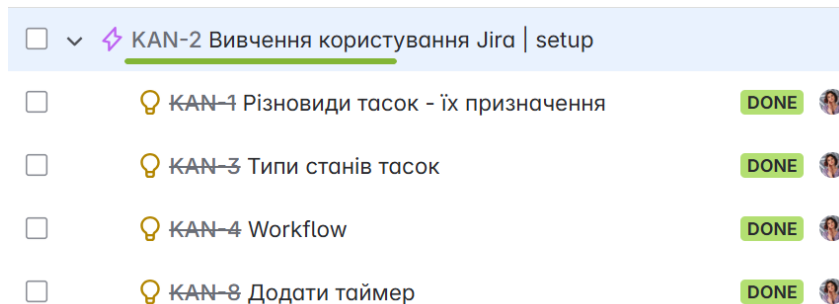


Рисунок 2.2.2 – Kan-2 та його вміст

Kan-1 має в собі задачу створення видів задач. Kan-3 має в собі задачу створення станів задач. Kan-4 має в собі задачу редакції “workflow”. Kan-8 має в собі задачу інтеграції відслідковування часу. (див. рис. 2.2.3)

Всі задачі з цього епіку були назначені та виконанні Куліченко С.Р.

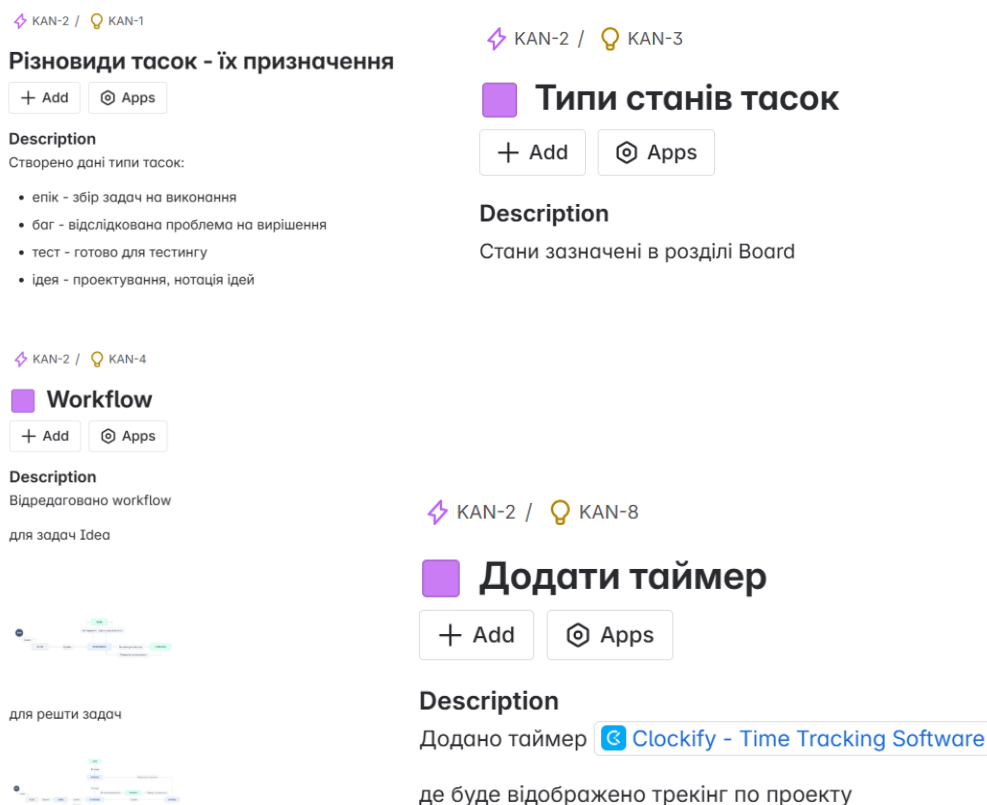


Рисунок 2.2.3 – Kan-1, Kan-3, Kan-4, Kan-8 та їх вміст

Kan-9 покриває собою створення «Github» репозиторіїв та їх інтеграція з «Atlassian | Jira» та «Unity». (див. рис. 2.2.4)

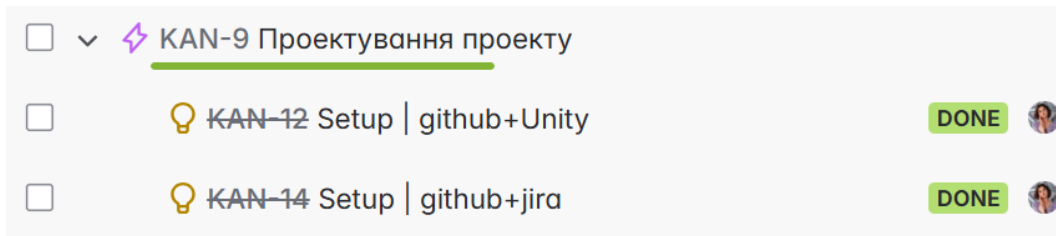


Рисунок 2.2.4 – Кан-9 та його вміст

Кан-12 має в собі задачу створення зв'язку «Github» і «Unity». Кан-14 має в собі задачу створення зв'язку «Github» і «Atlassian | Jira». (див. рис. 2.2.5)

Всі задачі з цього епіку були назначені та виконанні Куліченко С.Р.

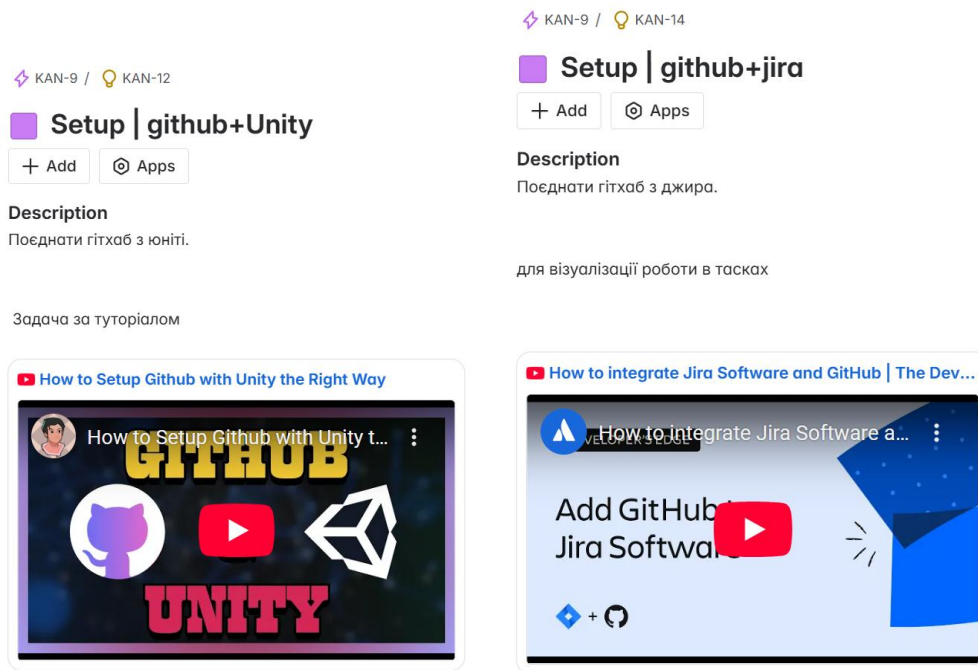


Рисунок 2.2.5 – Кан-12, Кан-14 та їх вміст

Кан-10 покриває собою етап проектування гри(див. рис. 2.2.6).

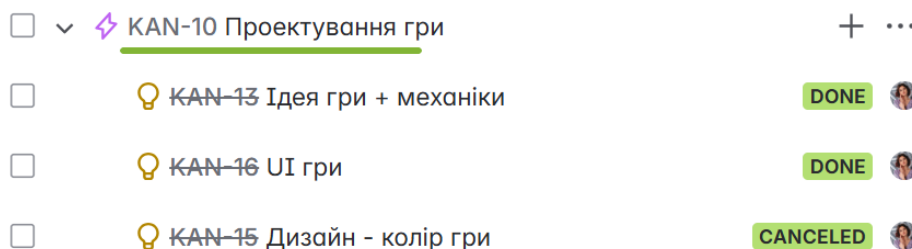


Рисунок 2.2.6 – Кан-10 та його вміст

Кан-13 має прописану логіку ігровим механікам та ідею ігрового процесу. Кан-16 має етапи проектування та розробки ігрового інтерфейсу. Кан-15 має в собі задачу розробки декількох кольорових тем для гри, що стала відміненою через недоцільність виконання. (див. рис. 2.2.7)

Всі задачі з цього епіку були поставлені Куліченко С.Р.

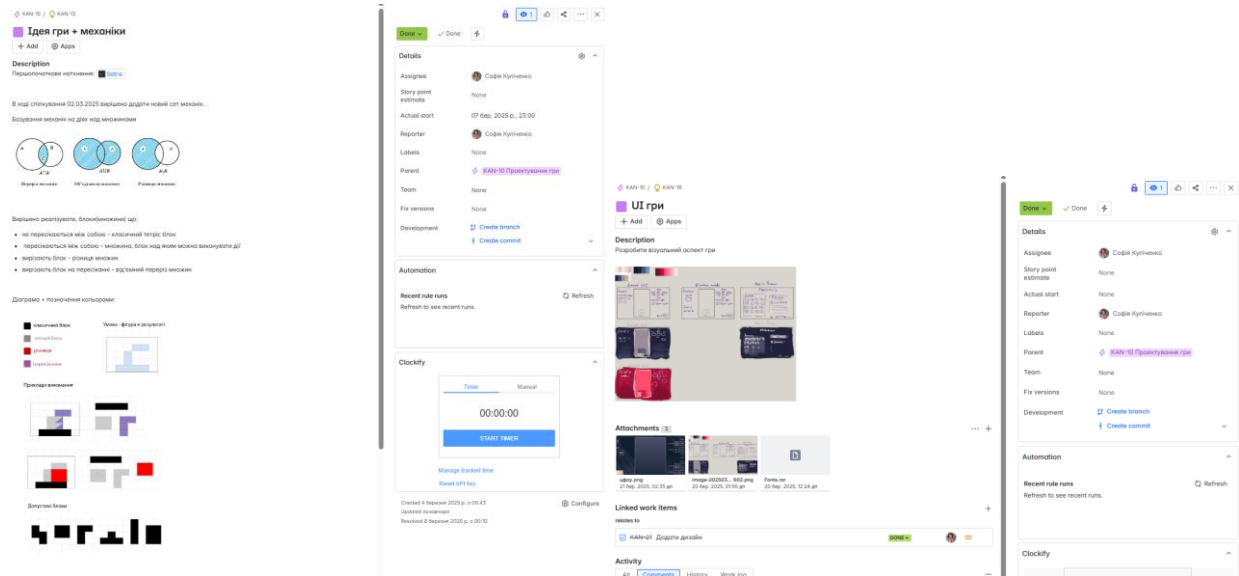


Рисунок 2.2.7 – Кан-13 та Кан-16 та їх вміст

Етап розробки гри через його об'ємність було поділено на три підходи. (див. рис. 2.2.8) Кожен етап мав задачу на тестування і подальше виправлення багу за виявлення такого під час тесту. Тільки після виправлення багу розробка переходила на наступну ступінь.

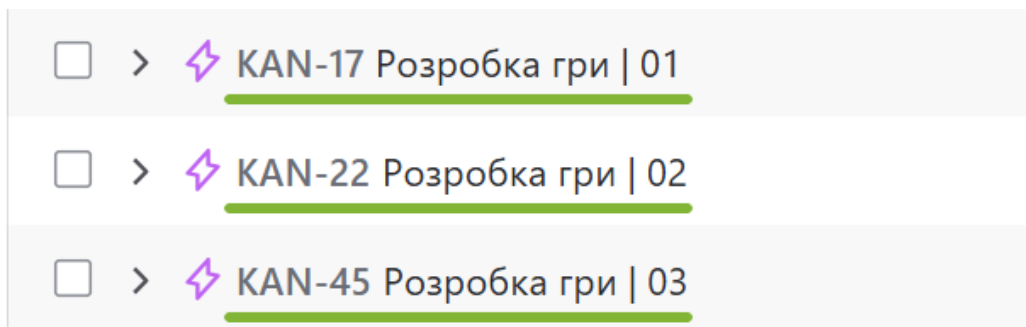


Рисунок 2.2.8– Три епіки для розробки гри

Кан-17 покриває собою перший етап розробки гри. (див. рис. 2.2.9)

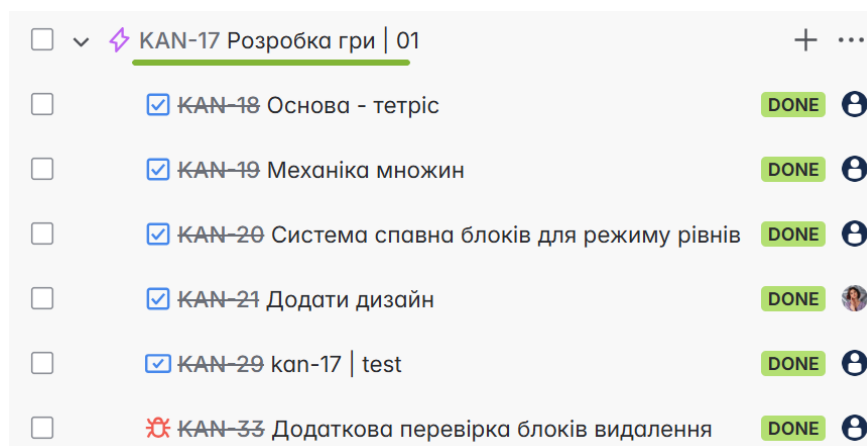


Рисунок 2.2.9– Кан-17 та його вміст

Кан-18 має в собі задачу розробки базової механіки гри «Тетріс». Кан-19 має в собі задачу розробки особливих механік гри запроєктованих в Кан-13. Кан-20 має в собі задачу розробки системи генерації блоків, які обирає користувач на сітці в лівій частині екрану. Кан-21 має в собі задачу інтеграції візуальної частини гри розроблену в Кан-16. Кан-29 є підсумковим тестом епіка Кан-17. Кан-33 є вихідною багою з Кан-17, після виправлення якої всі задачі епіку було переведено в статус виконано. (див. рис. 2.2.10)

Всі задачі з цього епіку назначала Куліченко С.Р.

Всі задачі з цього епіку виконував Оніщенко Є.О., окрім Кан-21 яку виконувала Куліченко С.Р.

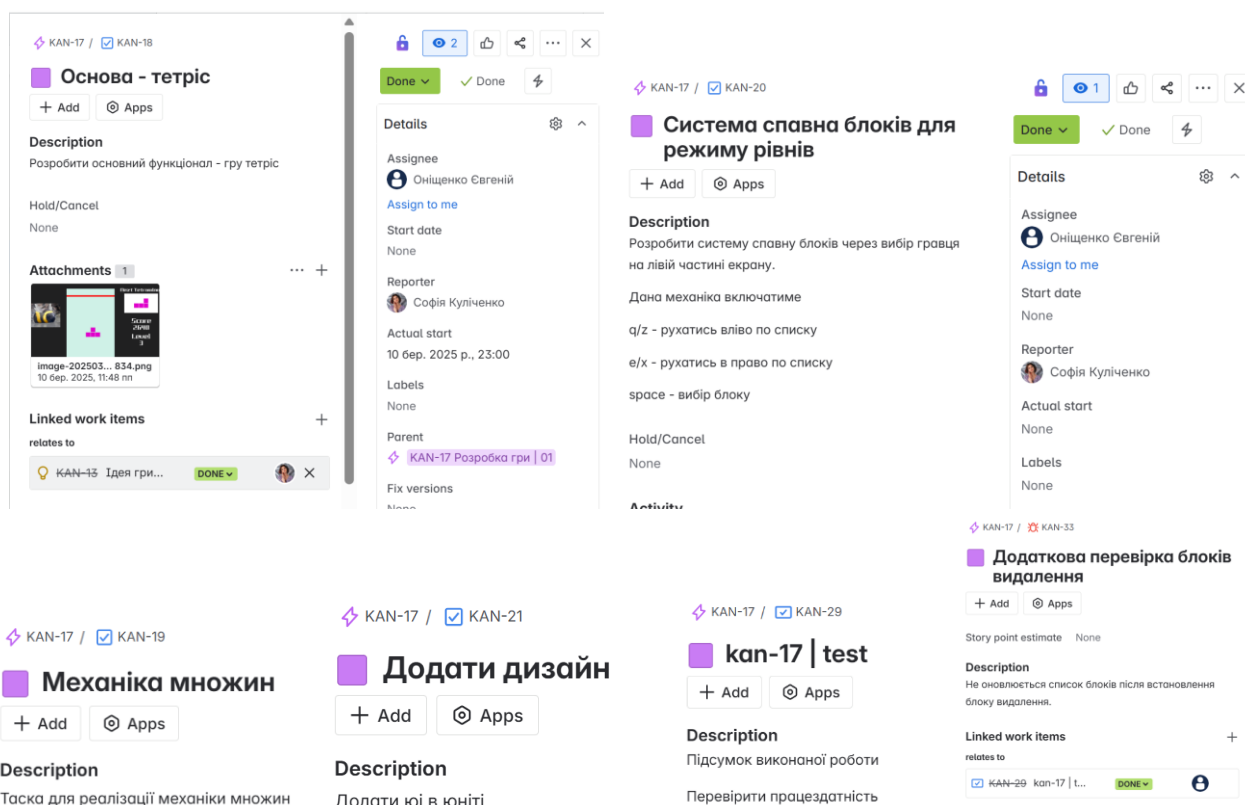


Рисунок 2.2.10 – Кан-18, Кан-19, Кан-20, Кан-21, Кан-29, Кан-33 та їх вміст

Кан-22 покриває собою другий етап розробки гри. (див. рис. 2.2.11)

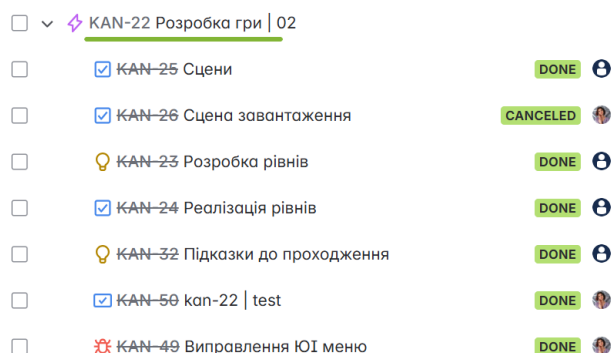


Рисунок 2.2.11 – Кан-22 та його вміст

Кан-25 має в собі задачу розподілення гри на три сцени: головного меню, режиму з рівнями, безкінечному режиму. Кан-26 має в собі задачу реалізації додаткової сцени для завантаження, задача скасована через недоцільність виконання. Кан-23 має в собі задачу розробки рівнів для відповідного режиму. Кан-24 має в собі задачу реалізації рівнів придуманих в Кан-23. Кан-32 має в собі задачу реалізації системи підказок щодо проходження рівнів. Кан-50 є підсумковим тестом епіка Кан-22. . Кан-49 є вихідною багою з Кан-50, після виправлення якої всі задачі епіку було переведено в статус виконано. (див. рис. 2.2.12)

Всі задачі з цього епіку назначала Куліченко С.Р.

Всі задачі з цього епіку виконував Оніщенко Є.О., окрім Кан-50 та Кан-49 яку виконувала Куліченко С.Р.

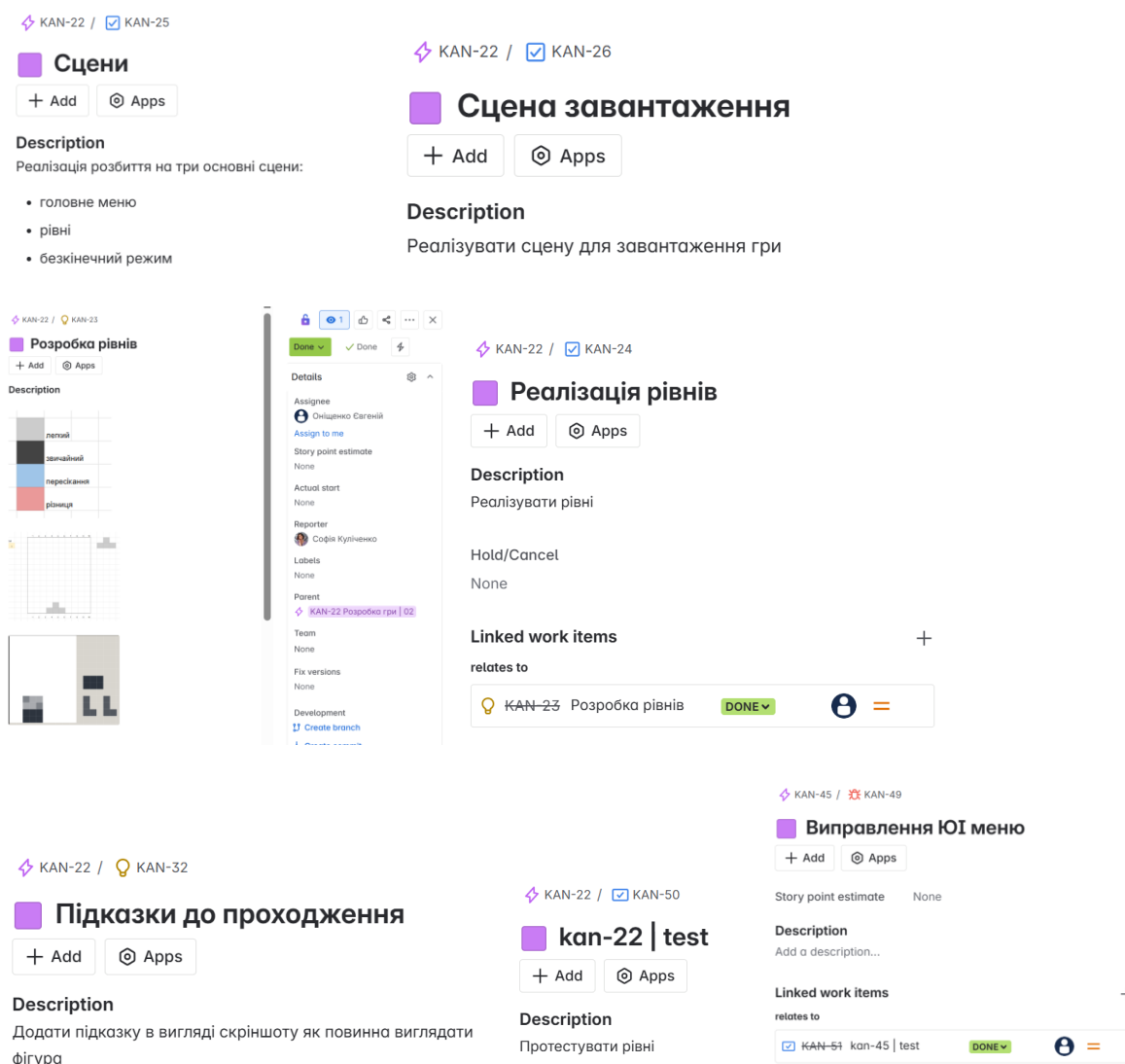


Рисунок 2.2.12 – Кан-25, Кан-26, Кан-23, Кан-24, Кан-32, Кан-49, Кан-50 та їх вміст

Кан-45 покриває собою третій і останній етап розробки гри. (див. рис. 2.2.13)

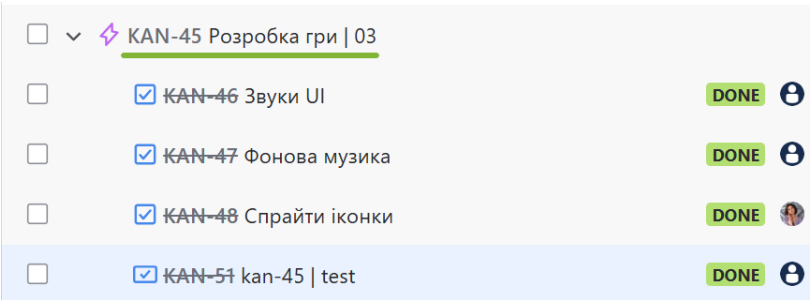


Рисунок 2.2.13 – Кан-45 та його вміст

Кан-46 має в собі задачу додавання звукового супроводу при клацанні на кнопку, встановленні блоку, завершені чи рестарту гри і т.д.(Детальніше в пункті про звуковий супровід). Кан-47 має в собі задачу додавання фонової музики. Кан-48 має в собі задачу розробки двох нових малюнків які не були застосовані в фінальній версії гри через їх недоцільність. Кан-51 є підсумковим тестом епіка Кан-45. (див. рис. 2.2.14)

Всі задачі з цього епіку назначала Куліченко С.Р., окрім Кан-48 яку назначив Оніщенко Є.О.

Всі задачі з цього епіку виконував Оніщенко Є.О., окрім Кан-48 які виконувала Куліченко С.Р.

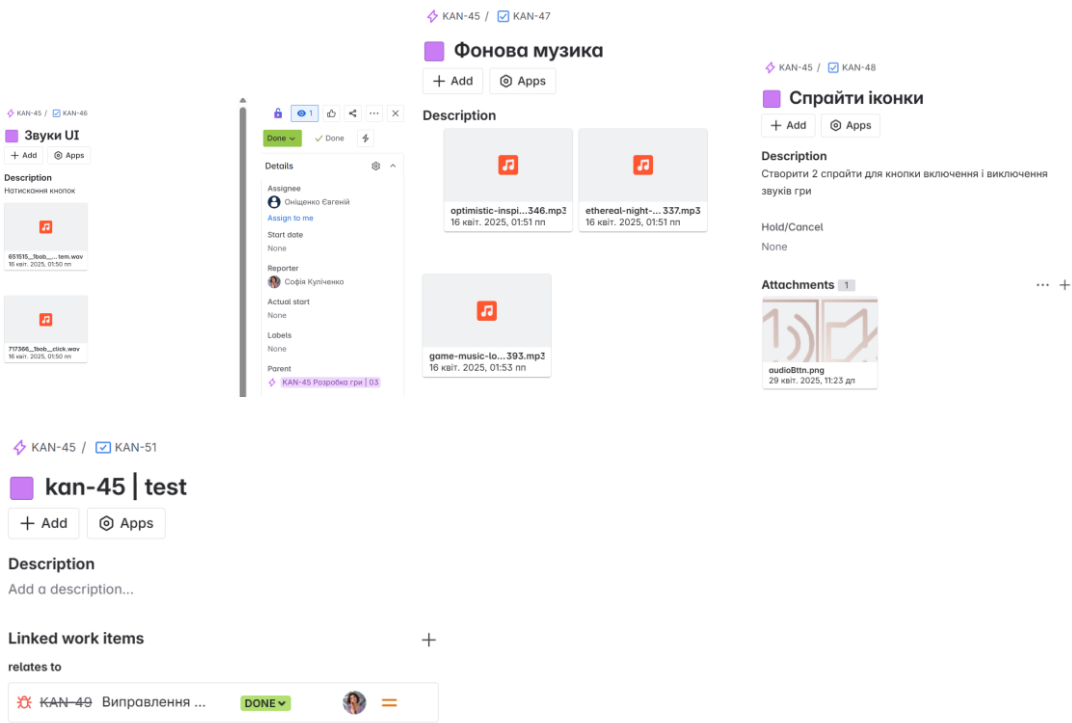


Рисунок 2.2.14 – Кан-46, Кан-47, Кан-48, Кан-51 та їх вміст

Кан-27 покриває собою етап розробки сайту. (див. рис. 2.2.15)

☐	▼	🔗 KAN-27 Сайт	
☐	💡	КАН-28 підготовка setup	DONE 👤
☐	✅	КАН-30 Створення сайту	DONE 👤
☐	✅	КАН-44 Зв'язок бд з сайтом	DONE 👤
☐	✅	КАН-55 Розробка фронтенду	DONE 👤
☐	✅	КАН-31 Вміст сайту	DONE 👤
☐	✅	КАН-43 Зв'язок юніті з сайтом	DONE 👤
☐	❌	КАН-56 Баг із зміною фокусу сайту	DONE 👤

Рисунок 2.2.15 – Кан-27 та його вміст

Кан-28 має в собі задачу створення репозиторію для збереження файлів сайту. Кан-30 має в собі задачу створення сайту на відповідному фреймворку. Кан-44 має в собі задачу розробки декількох рівнів веб застосунку для передачі інформації з БД. Кан-55 має в собі задачу розробки фронтенду. Кан-31 має в собі задачу проектування та розробки верхнього меню на сайті. Кан-43 має в собі задачу розроблення передачі даних з гри на сайт. Кан-56 є зафіксованою і виправленою багою. (див. рис. 2.2.16)

Всі задачі з цього епіку назначала Куліченко С.Р., окрім Кан-56 яку назначив Оніщенко Є.О.

Кан-55, Кан-43, Кан-56 виконував Оніщенко Є.О., Кан-28, Кан-30, Кан-44, Кан-31 виконувала Куліченко С.Р.

The image displays a Kanban board with seven task cards. Each card represents a task in a project, with a title, description, and activity section. The tasks are as follows:

- KAN-27 / KAN-28: підготовка | setup**
Description: Створити новий репозиторій для сайту. Надати доступ + під'єднати його до джирі
- KAN-27 / KAN-30: Створення сайту**
Description: Запустити сайт - зробити коміт в гіт
Activity: Вирішено використовувати vue + .net
- KAN-27 / KAN-31: Вміст сайту**
Description: Сайт складається з однієї сторінки яка має меню і основне вікно для гри
Activity: Розробити меню яке вміщує
- KAN-27 / KAN-44: Зв'язок бд з сайтом**
- KAN-27 / KAN-55: Розробка фронтенду**
- KAN-27 / KAN-43: Зв'язок юніті з сайтом**
Description: Налаштувати передачу даних з програми до сайту
Activity: Unity - Manual: Interaction with browser scripting
- KAN-27 / KAN-56: Баг із зміною фокусу сайту**
Description: При завантаженні WebGl бібліотека на сайт відзвучує з клавіатури повністю переключив до гри і робив неможливим відзвучування

Рисунок 2.2.16 – Кан-28, Кан-30, Кан-44, Кан-55, Кан-31, Кан-43, Кан-56 та їх вміст

Кап-34 покриває собою етап розробки БД. (див. рис. 2.2.17)

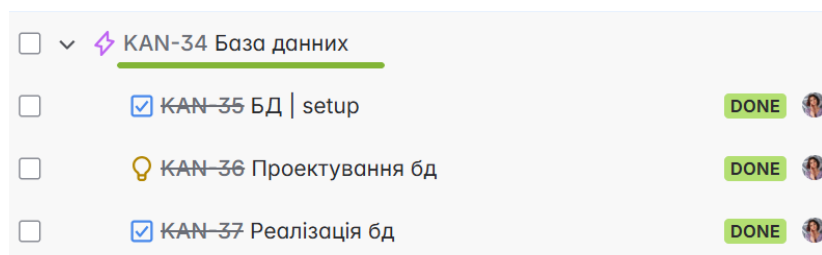


Рисунок 2.2.17 – Кап-34 та його вміст

Кап-35 має в собі задачу створення віддаленого підключення. Кап-36 має в собі задачу проектування БД. Кап-37 має в собі задачу реалізації БД. (див. рис. 2.2.18)

Всі задачі з цього епіку були назначені та виконанні Куліченко С.Р.

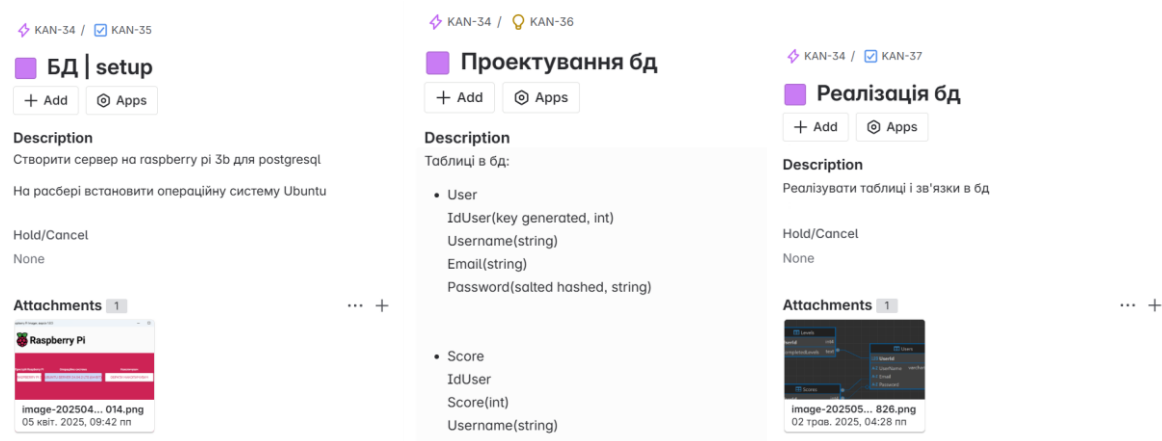


Рисунок 2.2.18 – Кап-34 та його вміст

Кап-38 була спланована додатково до вимог проекту та відмінена через нестачу ресурсу для виконання. (див. рис. 2.2.19)

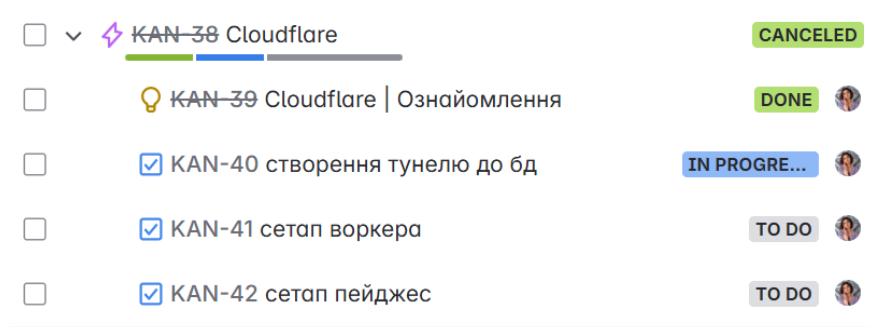


Рисунок 2.2.19 – Кап-38 та його вміст

Кап-27 покриває собою етап створення звіту та підготовки до захисту. (див. рис. 2.2.20)

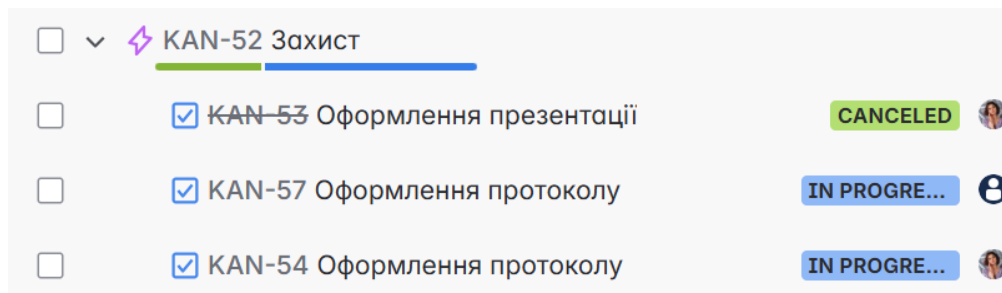


Рисунок 2.2.20 – Кан-34 та його вміст

Кан-53 була відмінена через недоцільність виконання. Кан-57 та Кан-54 слугують задачами для оформлення звіту. (див. рис. 2.2.21)

Всі задачі з цього епіку назначала Куліченко С.Р.

Кан-57 виконував Оніщенко Є.О., Кан-54 виконувала Куліченко С.Р.

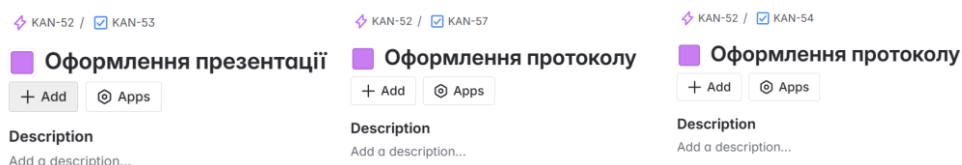


Рисунок 2.2.21 – Кан-53, Кан-57, Кан-54 та їх вміст

3. Підготовка інфраструктури

3.1. Допоміжні забезпечення

На початку було вирішено та затверджено список програмних забезпечень та пристроїв, які будуть задіяні в виконанні роботи

Дана таблиця охоплює список забезпечень, які були використані при проектуванні, створенні, тестування проекту. Опис ролі кожного забезпечення продемонстровано на табл. 3.1.1.

Таблиця 3.1.1 – Аналіз використання допоміжних забезпечень

Назва	Короткий опис
Atlassian Jira	Система організації та управління проєктів.
Github	Вебсервіс для спільної розробки програмного забезпечення.
Github desktop	Версія Github для десктопу
Clokify	Хмарний сервіс відстеження часу для команд.
Adobe Photoshop	Графічний редактор
Unity	Багатофункціональний інструмент для розроблення відеоігор і застосунків, і рушій, на якому вони працюють.

PostgreSQL	об'єктно-реляційна система керування базами даних(СУБД)
Putty	вільно розповсюджуваний клієнт для протоколів SSH, Telnet, rlogin і чистого TCP.
Dbeaver	SQL клієнт та інструмент управління базами даних.
.NET core 9	платформа для створення програм і веб-застосунків
Vue+ Vite	JavaScript-фреймворк, що використовує шаблон MVVM для створення інтерфейсів користувача
Visual studio	інтегроване середовище розробки програмного забезпечення
Visual studio code	редактор початкового коду
Postman	інструмент для тестування та розробки API

3.2. Інфраструктура - Jira

Для планування розподілення навантаження було використано програму управління проектами «Atlassian | Jira». За її допомогою було створено проект, який було відредаговано до відповідності вимог заданих курсовою роботою.

А саме додані нові:

- типи задач
- стани задач
- workflow

Створено дані типи задач(див. рис. 3.2.1 та табл 3.2.1):

Таблиця 3.2.1 – Типи задач

Назва	Призначення
епік(epic)	Етап реалізації проекту
таска(task)	задача на виконання
баг(bug)	відслідкована проблема на вирішення
тест(testing)	задача для тестування готового епіку
ідея(idea)	задача для фіксації етапів проектування, нотація ідей

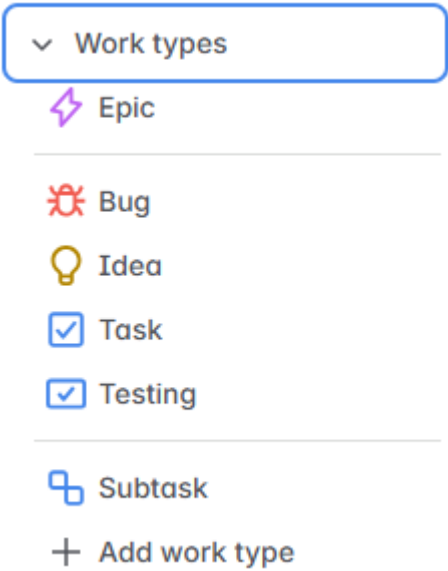


Рисунок 3.2.1 – Типи задач

Додано дані стани задач(див. рис. 3.2.2 та табл. 3.2.2):

Таблиця 3.2.2 – Стани задач

Назва	Призначення
To Do	Автоматично присвоюється при створення задачі
Opened	Взято на розгляд
Reopened	Відправлено назад на огляд
In Progress	В процесі виконання
Approval	Відправлено на погодження
Hold	Затримка з неможливості виконання
Done	Готово
Canceled	Не виконуватиметься

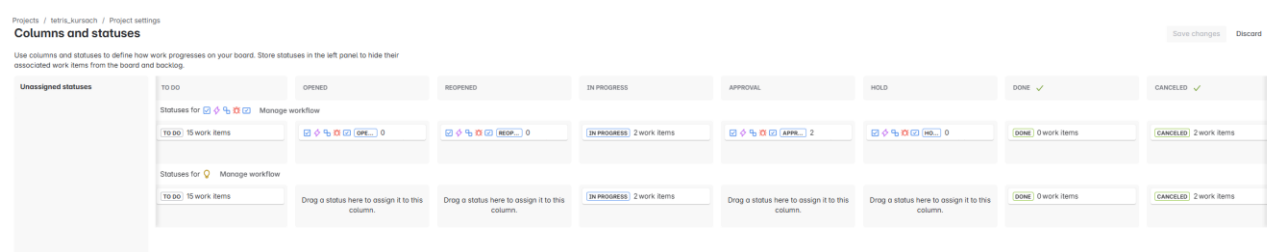
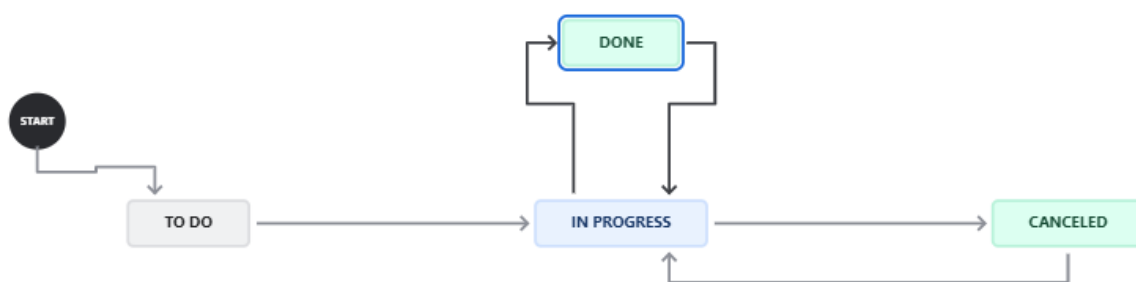
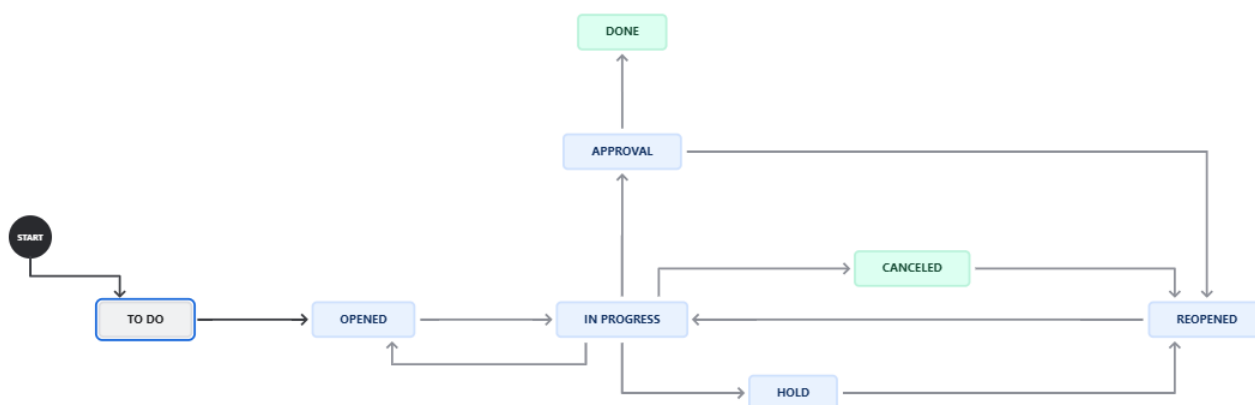


Рисунок 3.2.2 – Стани задач

Для задач проекту створено новий workflow(рис 3.2.3 а, б).



а



б

Рисунок 3.2.3 Workflow для а – задач Idea; б – решти задач.

Також було додано зв'язок до обох репозиторіїв та системи відслідковування часу.

Результатом даного пункту є налаштована інфраструктура «Atlassian | Jira». Дані налаштування були покриті епіком кап-2. Було витрачено близько три години на виконання. Питання закрито в планових термінах.

3.3. Часові витрати – Clockify

Для відстежування витраченого часу було використано програму «Clockify». За її допомогою є можливість переглянути витрачений час на завдання, по групуваний за різноманітними критеріями.

Для зручності використання «Clockify» підключений як плагін до «Atlassian | Jira». Це надало можливість фіксувати час витрачений на кожне завдання.

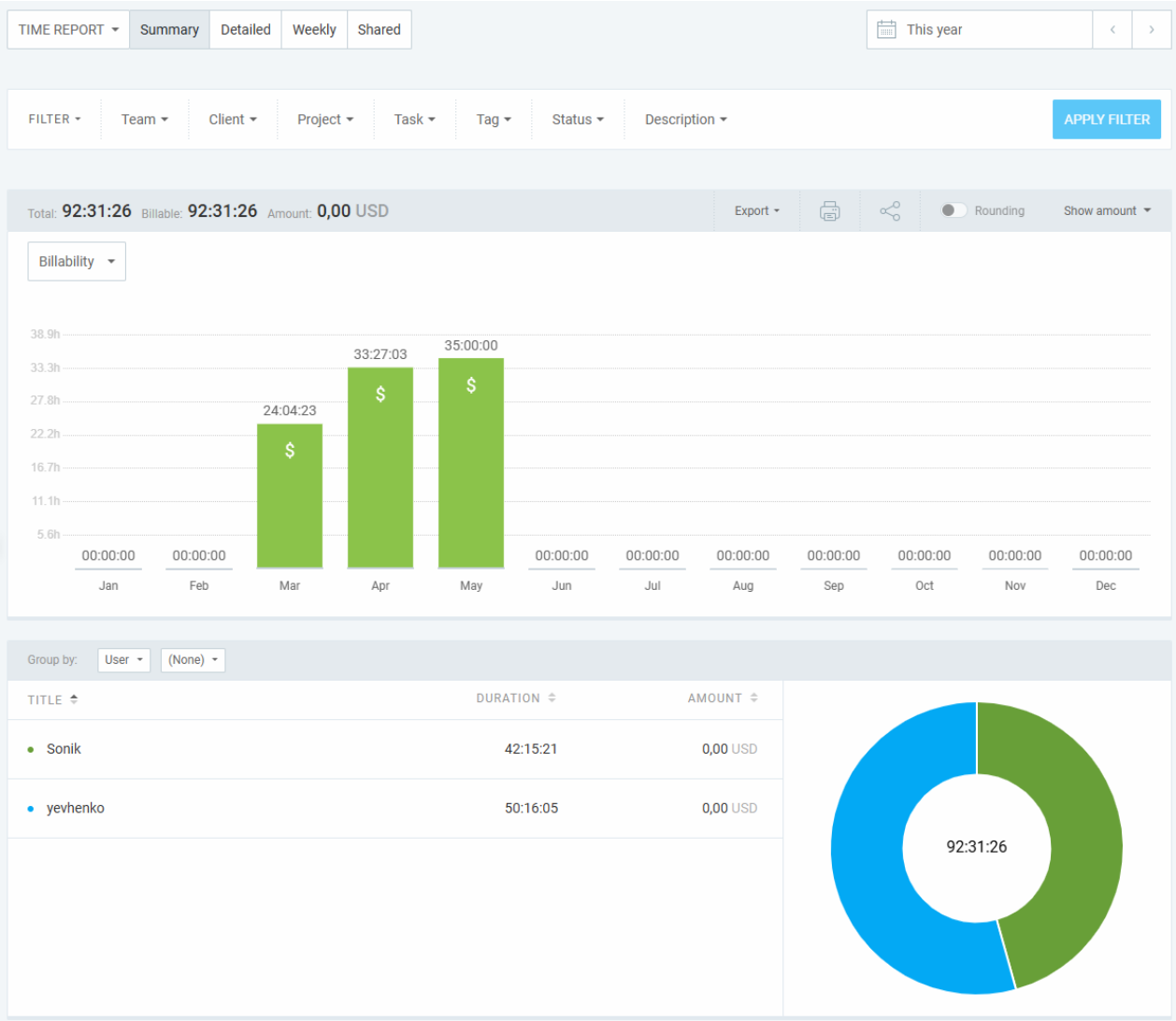


Рисунок 3.3.1 – Діаграма всього витраченого часу по учасникам

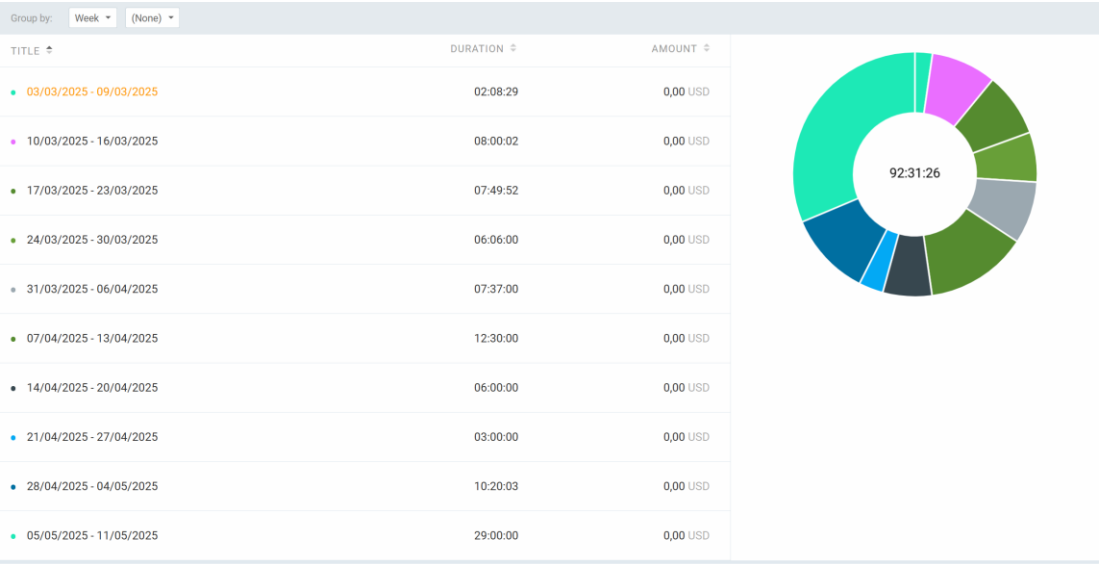


Рисунок 3.3.2 – Діаграма всього витраченого часу по тижнево

Total: 92:31:26 92:31:26 Amount: 0,00 USD					Export					Roundings					Show amount				
TIME ENTRY					AMOUNT					USER					TIME				
DURATION																			
[KAN-01] Kan-01 (test) • tetris_kursach KAN-01					Task					0,00 \$ Sonik -					14:58 14:58 Today				
[KAN-02] Створення проекту до гри • tetris_kursach KAN-02					Task					0,00 \$ Sonik -					14:53 17:53 Today				
[KAN-03] Завантаження файлів з сайту • tetris_kursach KAN-03					Task					0,00 \$ Sonik -					14:52 18:52 Today				
[KAN-04] Створення проекту • tetris_kursach KAN-04					Task					0,00 \$ Sonik -					14:52 20:52 Today				
[KAN-05] Завантаження файлів з сайту • tetris_kursach KAN-05					Task					0,00 \$ yehenko -					12:58 17:58 Today				
[KAN-06] Розробка проекту • tetris_kursach KAN-06					Task					0,00 \$ yehenko -					12:54 18:54 Today				
[KAN-07] Завантаження файлів з сайту • tetris_kursach KAN-07					Task					0,00 \$ yehenko -					12:54 19:54 Today				
[KAN-08] Завантаження файлів з сайту • tetris_kursach KAN-08					Task					0,00 \$ Sonik -					14:52 16:52 02/04/2023				
[KAN-09] Завантаження файлів з сайту • tetris_kursach KAN-09					Task					0,00 \$ Sonik -					12:25 16:25 01/04/2023				
[KAN-10] Завантаження файлів з сайту • tetris_kursach KAN-10					Task					0,00 \$ yehenko -					11:24 11:24 29/04/2023				
[KAN-11] Створення проекту • tetris_kursach KAN-11					Task					0,00 \$ Sonik -					11:05 11:05 29/04/2023				
[KAN-12] Завантаження файлів з сайту • tetris_kursach KAN-12					Task					0,00 \$ Sonik -					10:47 17:47 29/04/2023				
[KAN-13] Завантаження файлів з сайту • tetris_kursach KAN-13					Task					0,00 \$ Sonik -					11:22 12:22 29/04/2023				
[KAN-14] Завантаження файлів з сайту • tetris_kursach KAN-14					Task					0,00 \$ yehenko -					11:26 12:26 02/04/2023				
[KAN-15] Завантаження файлів з сайту • tetris_kursach KAN-15					Task					0,00 \$ yehenko -					11:24 17:24 29/04/2023				
[KAN-16] Завантаження файлів з сайту • tetris_kursach KAN-16					Task					0,00 \$ Sonik -					22:00 00:00 11/04/2023				
[KAN-17] Створення проекту • tetris_kursach KAN-17					Task					0,00 \$ Sonik -					16:00 18:00 11/04/2023				
[KAN-18] Створення проекту • tetris_kursach KAN-18					Task					0,00 \$ Sonik -					16:05 18:05 10/04/2023				
[KAN-19] Створення проекту • tetris_kursach KAN-19					Task					0,00 \$ yehenko -					22:08 22:38 29/04/2023				
[KAN-20] Завантаження файлів з сайту • tetris_kursach KAN-20					Task					0,00 \$ yehenko -					00:30:00				

Рисунок 3.3.3 – Діаграма всього витраченого часу по задачам

Результатом даного пункту є налаштована інфраструктура «Clockify». Дані налаштування були покриті епіком kan-2. Було витрачено близько пів години на виконання. Питання закрито в планових термінах.

3.4. Контроль версій – Git

Для контролю і управління версіями проекту було використано платформу «Github».

Оскільки дана робота поділяється на декілька об'ємних етапів було прийнято рішення застосовувати два окремих репозиторія. Детальніше про кожен в таблиці 3.4.1.

Таблиця 3.4.1 – Репозиторії

Назва	Призначення	Посилання
tetris_kursach	Для утримання коду гри до проекту Unity	https://github.com/SonikLvl/tetris_kursach.git
web_kursachsem4	Для утримання коду сайту	https://github.com/SonikLvl/web_kursachsem4.git

Це зроблено оскільки проект «Unity» легше і доцільніше відкривати як окрему папку яку ми копіюємо з репозиторію. В свою чергу він має відповідний файл .gitignore(файл призначений для ігнорування тимчасових файлів які автоматично створюються «Unity»).

За аналогічним принципом зроблено і другий репозиторій, який вміщає в собі результат роботи першого – а саме “build”(компільована версія проекту готового до ігрового процесу).

Результатом є два репозиторії готові до використання. Дані налаштування були покриті епіком kan-9. Було витрачено близько дві години на виконання. Питання закрито в планових термінах.

4. Гра

4.1. Проектування гри

За основу гри було прийняте рішення взяти відеогру-головоломку «Тетріс».

В ході спілкування 02.03.2025 вирішено додати нове доповнення до старих механік. Механіки були створені на базі дій над множинами(див. рис. 4.1.1).

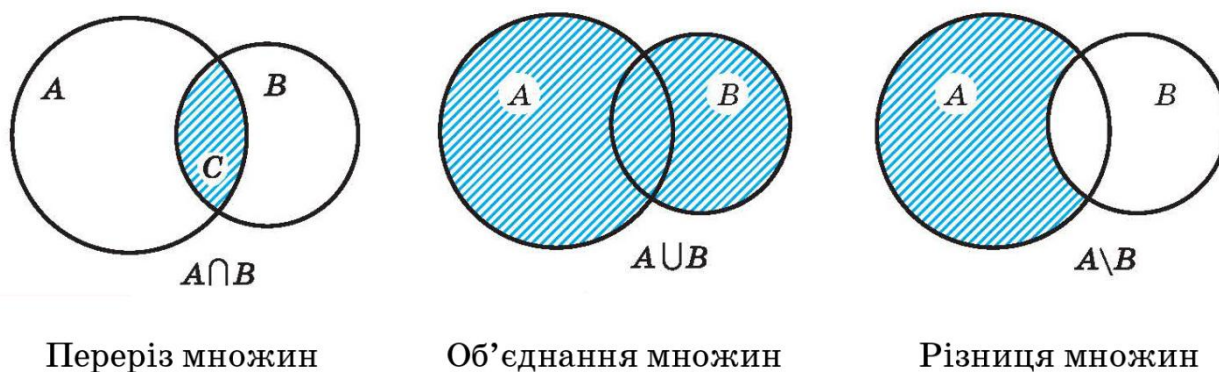


Рисунок 4.1.1 – Дії над множинами

Вирішено реалізувати, блоки(множини) зазначені в табл 3.1.1.

Таблиця 4.1.1 – Блоки в грі

Назва	Взаємодія	Дія з множинами
Класичний блок	Блок який не має особливої взаємодії з іншими. Поводить себе як класичний тетріс блок	Не є множиною
Легкий блок	Заповнена множина.	Об'єднання
Блок різниця	Пуста множина.	Різниця
Блок пересікання	Заповнена множина, стає пустою на перерізі	Від'ємний переріз

Приклад взаємодії наведено на діаграмі(рис. 4.1.2)

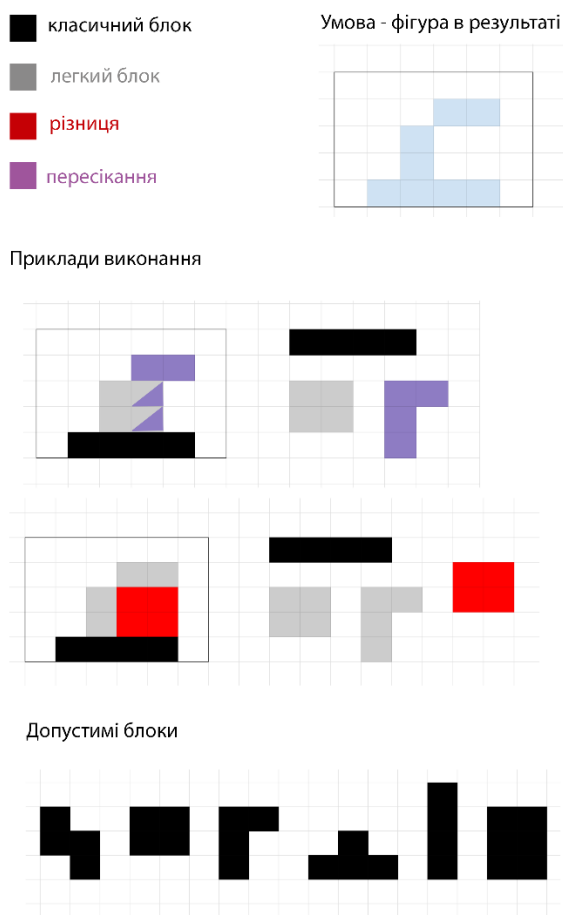


Рисунок 4.1.2 – Блоки та їх взаємодія

Також на даному етапі було вирішено створити три ігрові сцени:

- головне меню
- режим для рівнів
- безкінечний режим

Особливі механіки були застосовані тільки для рівневого режиму, безкінечна гра являє собою звичайну гру «Тетріс» з рахунком.

Результатом даного пункту є запроектована логіка гри. Даний пункт покрила задача кап-13. Було витрачено близько 45 хвилин на виконання. Питання закрито в планових термінах.

4.2. Візуальна частина гри

Інтерфейс гри було виконано в інтуїтивно зрозумілому і простому стилі. Було прийнято рішення не перевантажувати візуальний аспект і зосередитись на використанні лімітованої кольорової палетки та контрастному виділенні важливої для гравця інформації.

Першим етапом було зроблено декілька замальовок та обрана палітра кольорів(див. рис. 4.2.1).



Рисунок 4.2.1 – Замальовки інтерфейсу гри. Представлені всі три сцени

Після затвердження кольорової теми наступним етапом стала вимальовка грових спрайтів (“sprite” – термін з англійської, який описує 2д зображення чи анімацію використану в грі) та подальше створення спрайт-sheet (“sprite-sheet” – термін з англійської, який описує збірку менших зображень). Це необхідно для меншого навантаження системи – замість рендерингу десятка малих файлів по черзі, доцільніше зробити це один раз з великим файлом.

На рис. 4.2.2 зображено етап розробки і точного підбору кольорів для лаконічного вигляду в грі.



Рисунок 4.2.2 – Фінальне затвердження дизайну

Таблиця 4.2.1 демонструє всі створені “sprite-sheets” та спрайти завантажені в гру.

Таблиця 4.2.1 - Спрайти і їх призначення

			
Фоновий малюнок		Головна панель	
			
Відображення наступного блоку Блок пересікання		Відображення наступного блоку Блок різниці	

Відображення наступного блоку Легкий блок	Відображення наступного блоку Класичний блок
Фрагменти для блоків	Рухливе коло на фоні
Кнопка звуку	Відображення наступного блоку Безкінечний режим

Візуальний аспект було адаптовано під будь-які екрани комп'ютерів та ноутбуків. За потреби фоновий малюнок разом з колом розтягуються, а панель разом з іншими елементами на екрані, зберігаючи масштабування і орієнтуючись на фон, приймає необхідні розміри.

Також заради лаконічності дизайну було використано особливий шрифт.(рис. 4.2.3)

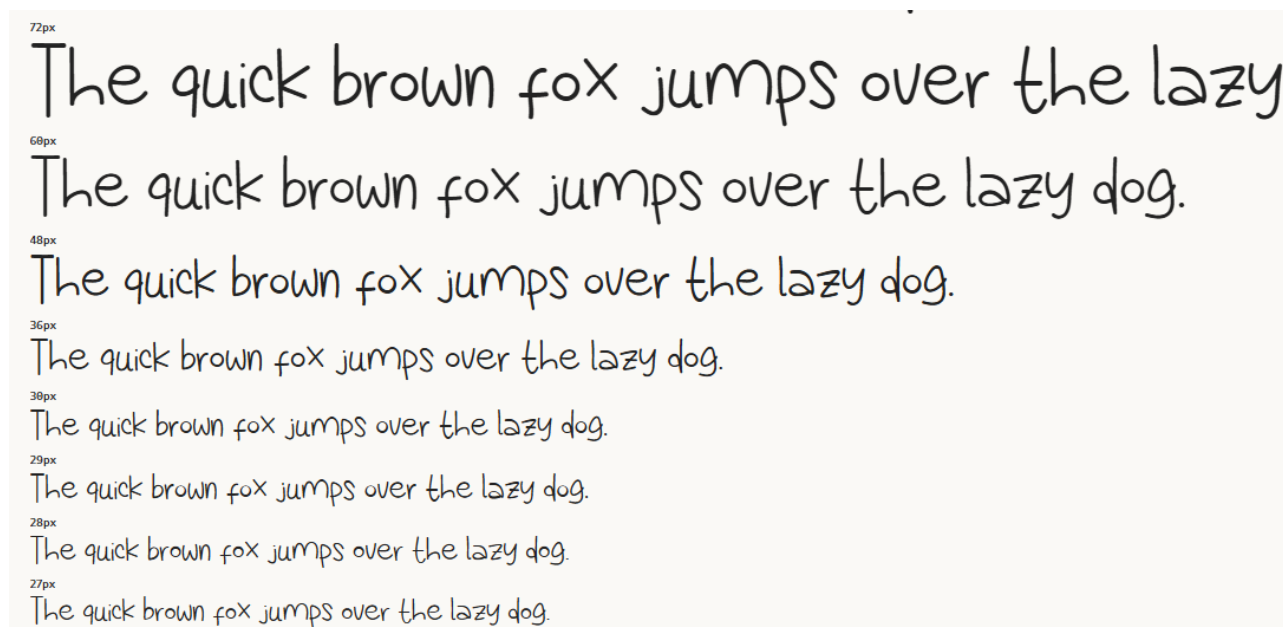
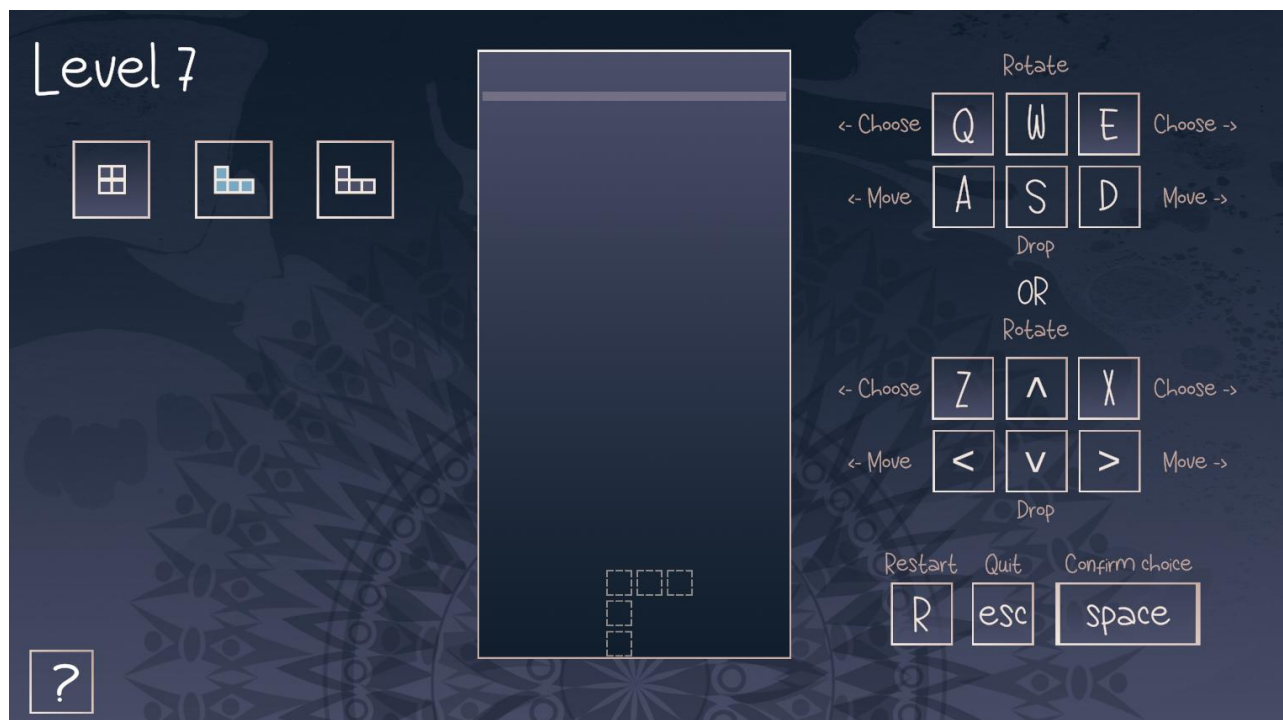


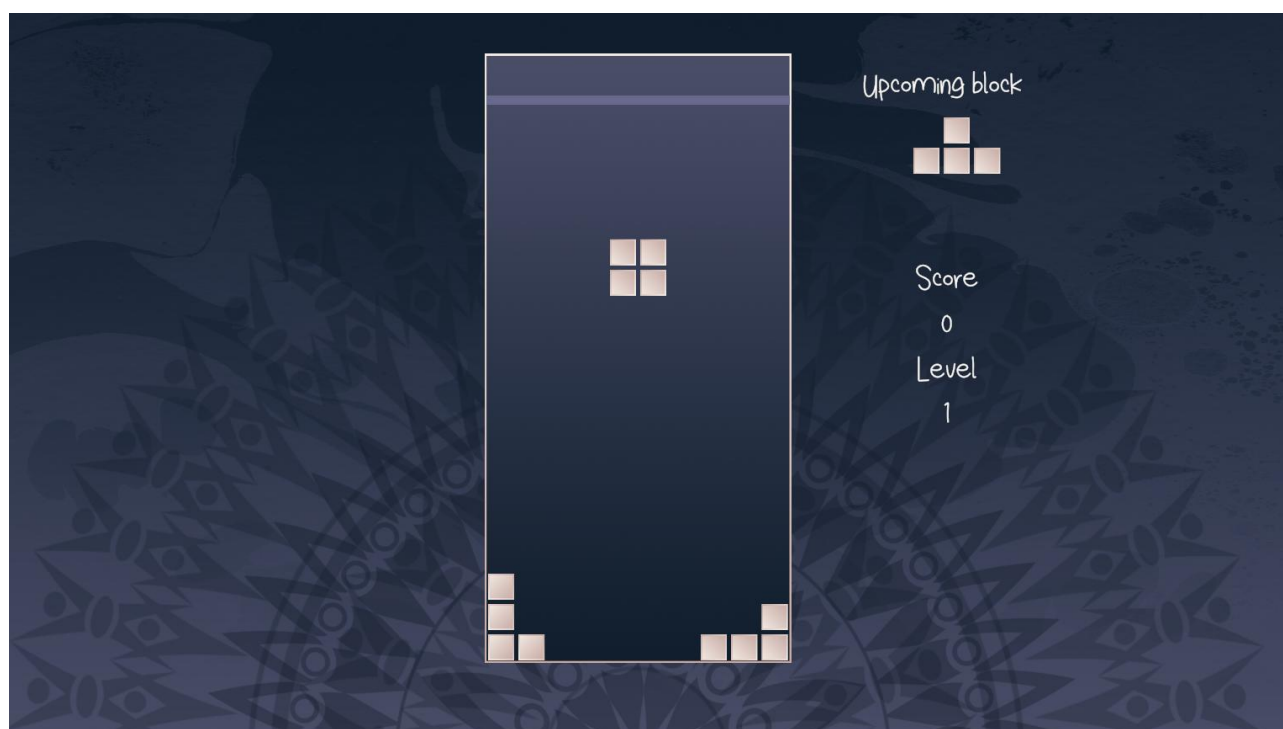
Рисунок 4.2.3 – Демонстрація шрифту Like Slim



а



б



в

Рисунок 4.2.4 – Фінальний результат. а - сцена меню; б – сцена рівнів; в – сцена безкінечного режиму.

Результатом даного пункту є готовий розроблений візуальний аспект програми. Даний пункт покрила задача кап-16. Було витрачено близько дві години на виконання. Питання закрито в планових термінах.

4.3. Безкінечний ігровий режим

Безкінечний режим має основні механіки гри «Тетріс» з підбиванням рахунку при розбитті блоків зібраних в рядок. Також, в залежності від кількості зруйнованих рядків, вираховується рівень. Також показується наступний блок на екрані для подальшої стратегії гравця. З рівнем блоки падають швидше.

Скрипти в грі відіграють необхідну механіку орієнтуючись на сцену.

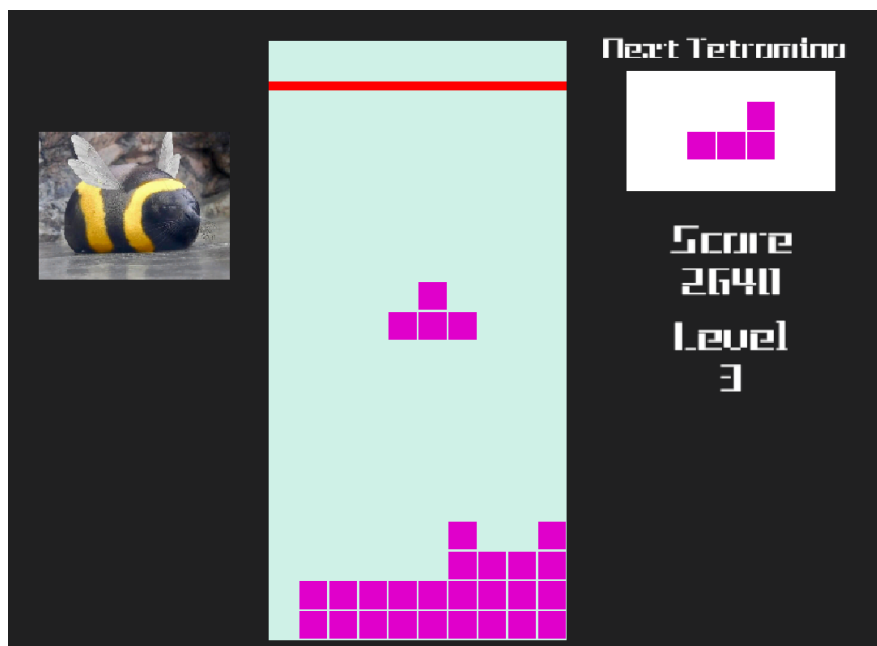


Рисунок 4.3.1 – Рання стадія розробки гри

4.4. Рівневий ігровий режим

Ціллю рівня є поставити всі блоки так щоб залишилась лише фігура показана пунктирним силуетом на панелі.

Рівневий режим має 20 рівнів доступних для проходження гравцем. З головного меню є можливість обрати будь-який рівень для проходження, після виграшу гравця переводить на наступний рівень, при програшу є можливість перезапустити рівень і почати заново, або вийти в меню і обрати інший.

Гравцю представлено інтуїтивно зрозумілий інтерфейс з описом механік в правій частині екрану і номером рівня та панелі вибору блоків в лівій. Все управління в грі виконується за допомогою клавіатури. В лівому нижньому кутку також є кнопка для демонстрації підказки. Підказка має під собою приклад фінального вигляду фігури.(див. рис. 4.4.1)

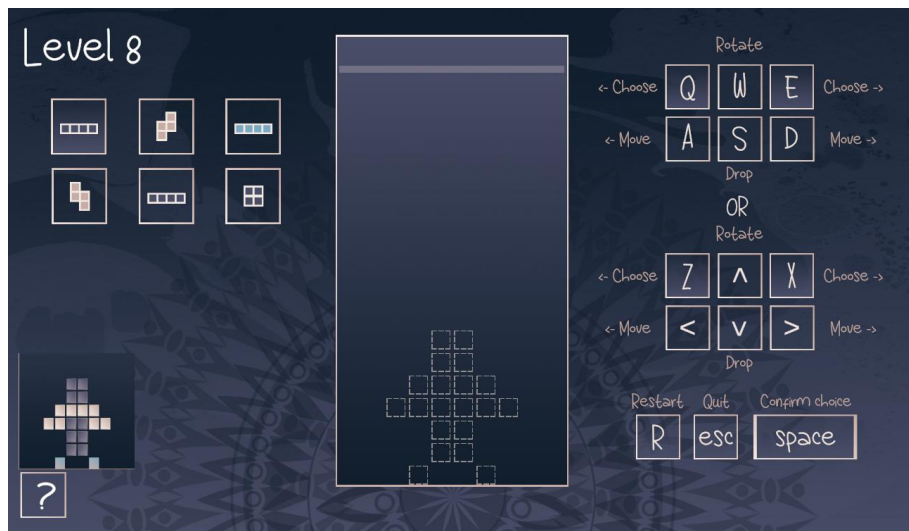


Рисунок 4.4.1 – Підказка до рівня

4.5. Код до гри

Даний пункт виконував інший учасник – Оніщенко Є.О. Для деталізації див. звіт «Курсова_робота_IO-32_Оніщенко.docx».

4.6. Звуковий супровід

Даний пункт виконував інший учасник – Оніщенко Є.О. Для деталізації див. звіт «Курсова_робота_IO-32_Оніщенко.docx».

4.7. Інфраструктура - «Atlassian | Jira»

Було створено декілька гілок в репозиторії доступ до яких забезпечено з проекту «Jira».(див. рис. 4.7.3)

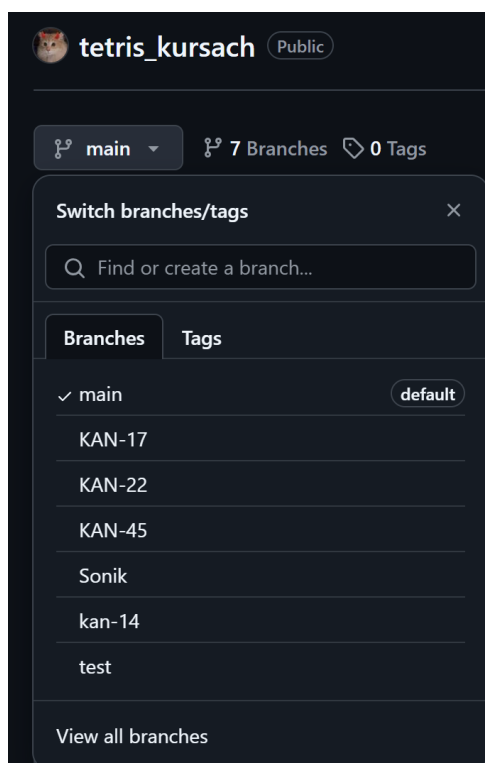


Рисунок 4.7.3 – Гілки репозиторію «tetris_kursach»

Це необхідно для правильного відображення кожного коміту до відповідного завдання.(див. рис. 4.7.4)

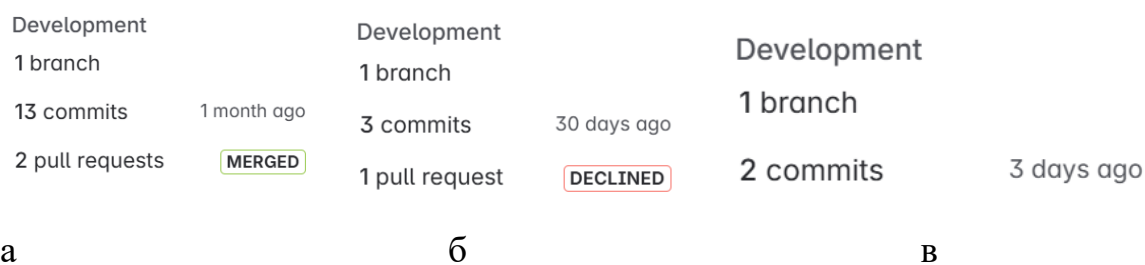


Рисунок 4.7.4 – Коміти до епиків. а – kan-17; б – kan-22; в – kan-45.

5. База даних

5.1. Проектування бази даних

База створюється для збереження даних користувача для входу на сайт та запису його ігрового рахунку при використанні безкінечного режиму.

Для задовільнення вимог створені дві таблиці:

Users

Назва стовпцю	Тип даних	Призначення
UserId	число	Ідентифікатор користувача
UserName	текст	Ім'я введене користувачем
Email	текст	Імейл введений користувачем
Password	текст	Хеш пароля введеного користувачем

Scores

Назва стовпцю	Тип даних	Призначення
UserId	число	Ідентифікатор користувача
ScoreValue	число	Найвищий отриманий користувачем рахунок
UserName	текст	Ім'я для підпису користувача в таблиці лідерів

Отже, всі дані з таблиці «Users» будуть надходити і запитуватись тільки на самому сайті. З таблиці «Scores» «ScoreValue» буде надходити з гри і запитуватись з сайту, інші значення в ній є іноземними ключами з «Users».

Детальніше про таблицю «Users».

UserId – це первинний ключ і унікальне автоматично згенероване обов’язкове число

UserName – це унікальне обов’язкове текстове поле

Email - це унікальне обов’язкове текстове поле

Password - це унікальне обов’язкове текстове поле. Пароль зберігається в криптованому вигляді (“hashed and salted” – спеціальний алгоритм шифрування для паролів)

Детальніше про таблицю «Scores».

UserId – це первинний і іноземний ключ з таблиці «Users»

ScoreValue – це обов’язкове числове поле, яке при створенні отримує значення 0

UserName – це копія значення «UserName» з таблиці «Users»

Діаграма реалізованої БД показана на рис. 5.1.1

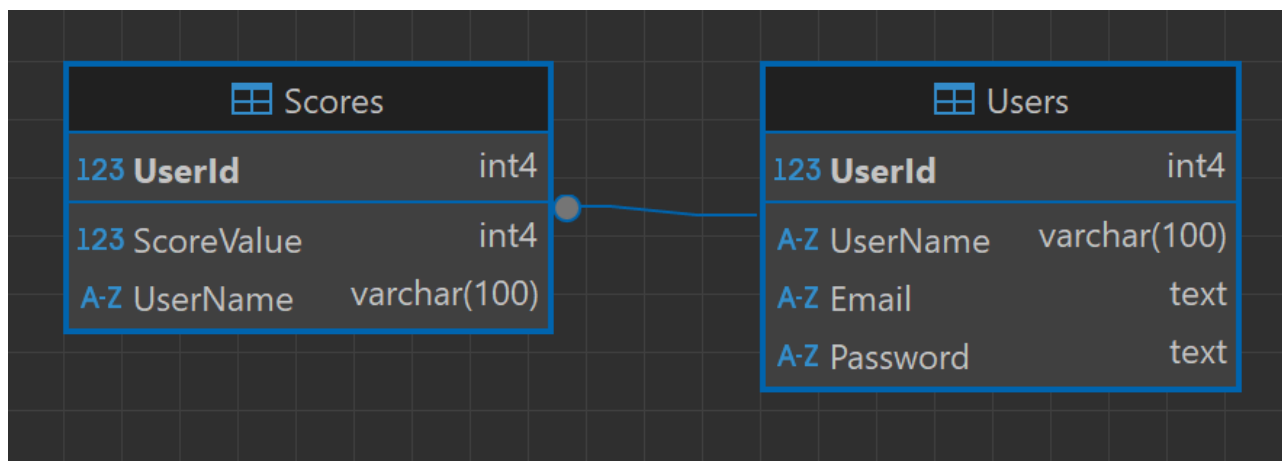


Рисунок 5.1.1 – Діаграма реалізованої БД

Результатом даного пункту є запроектована БД. Даний пункт покрила задача кап-36. Було витрачено приблизно пів години на виконання. Питання закрито в планових термінах.

5.2. Технічна підготовка віддаленого підключення

В межах даного проекту було реалізовано базу даних на окремому одноплатному комп’ютері Raspberry Pi 3b, до якого можна отримати підключення з локальної мережі.

З технічного обладнання було застосовано:

- одноплатний комп'ютер Raspberry Pi 3b
- мікро СД на 32 мб
- оригінальний зарядний провід
- підключення до інтернету
- монітор
- клавіатура

Для початку роботи потрібно встановити на мікро СД “image” операційної системи. Використовуємо Raspberry Pi imager для цієї задачі.(див. рис. 5.2.1)

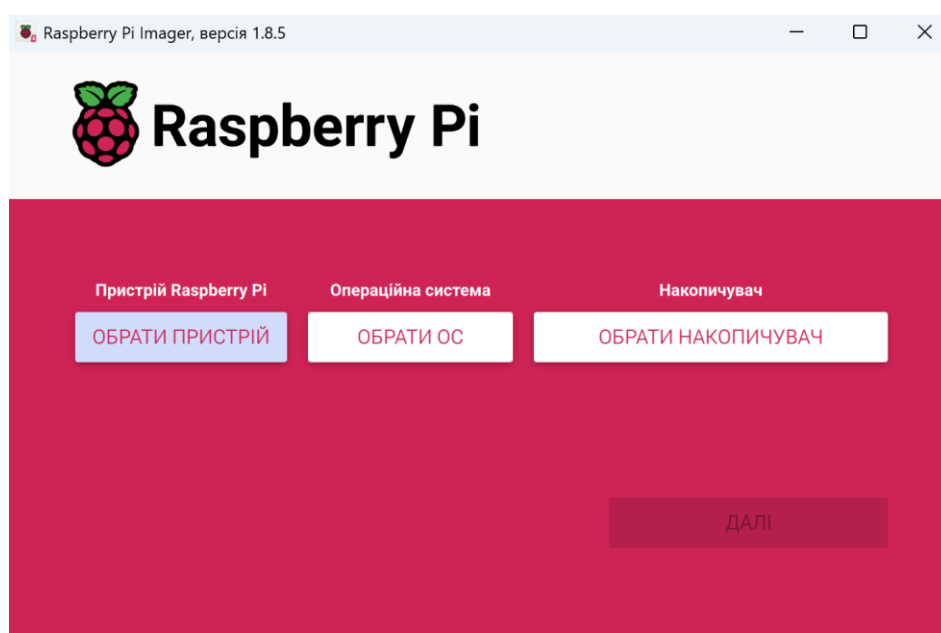
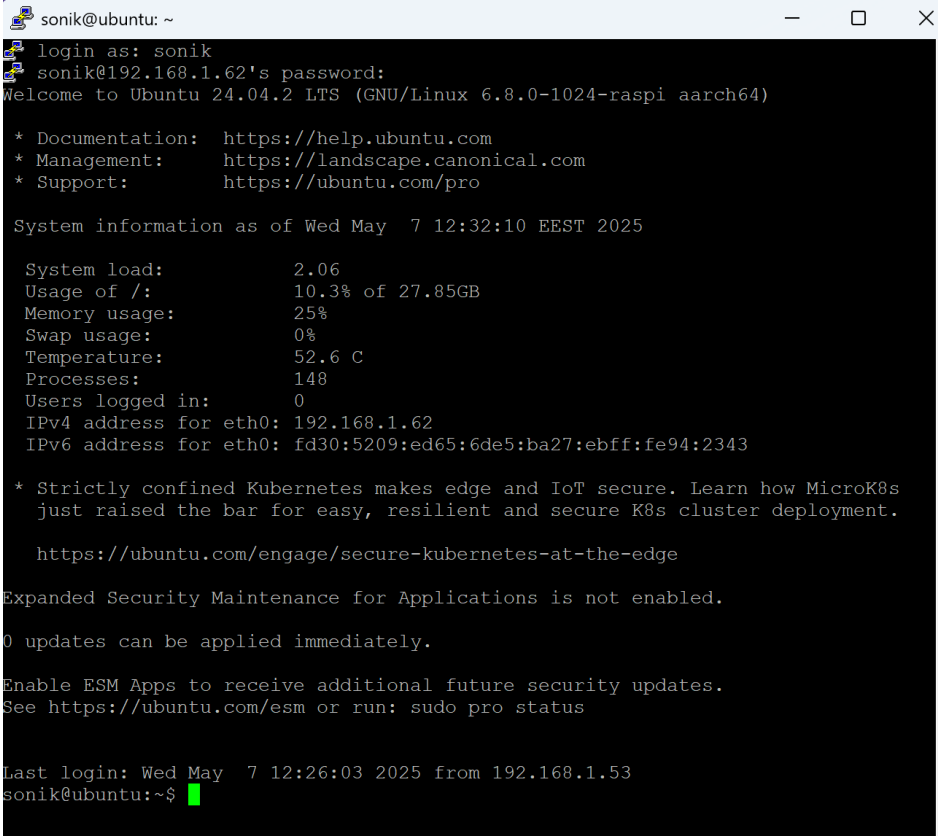


Рисунок 5.2.1 – Графічний інтерфейс програми Raspberry Pi imager

За допомогою першої кнопки «обрати пристрій» легко обрати точну версію комп'ютера Raspberry яка наявна в нас. Далі «обрати ос» вибираємо серверну Ubuntu 24.04.2 LTS. І останнім пунктом обираємо накопичувач куди завантажуюмо операційну систему.

Також перед завантаженням ми маємо можливість створити юзера з паролем.

Після цього підключаємо все обладнання і заходимо в систему. Для демонстрації підключення в даній курсовій роботі використовуємо програму для роботи з підключеннями по мережі «Putty».(див. рис. 5.2.2)



```

sonik@ubuntu: ~
login as: sonik
sonik@192.168.1.62's password:
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1024-raspi aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed May  7 12:32:10 EEST 2025

System load:          2.06
Usage of /:           10.3% of 27.85GB
Memory usage:        25%
Swap usage:          0%
Temperature:         52.6 C
Processes:           148
Users logged in:      0
IPv4 address for eth0: 192.168.1.62
IPv6 address for eth0: fd30:5209:ed65:6de5:ba27:ebff:fe94:2343

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed May  7 12:26:03 2025 from 192.168.1.53
sonik@ubuntu:~$

```

Рисунок 5.2.2 – Вхід в систему за допомогою «Putty»

Для перевірки оновлень і для їх встановлення раним команди:

```
sudo apt update
```

```
sudo apt upgrade
```

Для даної роботи було прийнято рішення використовувати СУБД – PostgreSQL.

Тому наступним кроком встановлюємо відповідні пакети:

```
sudo apt install postgresql
```

Після встановлення ми автоматично маємо створеного користувача-адміністратора postgres але в цілях безпеки створимо нового користувача.

Заходимо під користувачем СУДБ і створюємо нового:

```
sudo su postgres
```

```
createuser <USERNAME> -P -interactive
```

```
sonik@ubuntu:~$ sudo su postgres
[sudo] password for sonik:
Sorry, try again.
[sudo] password for sonik:
postgres@ubuntu:/home/sonik$ cd ~
postgres@ubuntu:~$ createuser showcaseuser -P --interactive
Enter password for new role:
Enter it again:
Shall the new role be a superuser? (y/n) █
```

Рисунок 5.2.3 – Створення користувача в СУБД

Тепер ми маємо можливість створити базу і увійти режим редагування:

psql

CREATE DATABASE <USERNAME>;

Для виходу використовуємо команду exit.

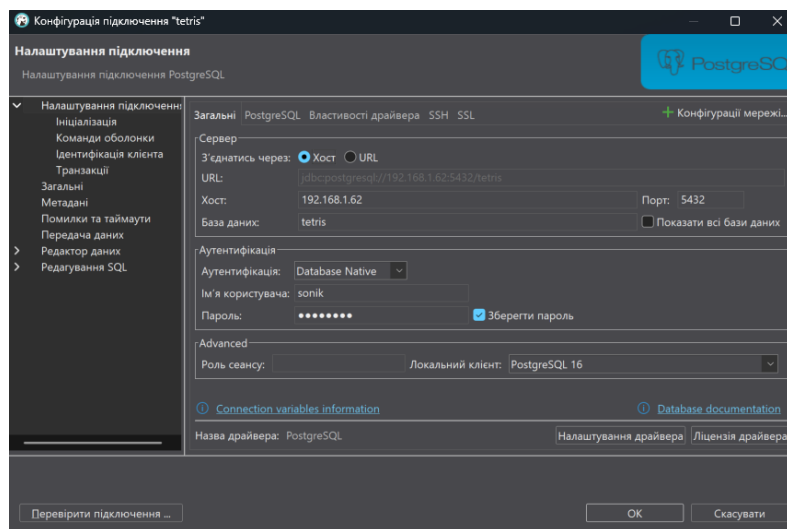
```
sonik@ubuntu:~$ sudo su postgres
postgres@ubuntu:/home/sonik$ psql
psql (16.8 (Ubuntu 16.8-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# █
```

Рисунок 5.2.3 – Робота з БД

Тепер Raspberry Pi налаштовано і ми повинні забезпечити підключення до інтернету і заряду для подальшої роботи, все інше більше не потрібно.

Для перевірки з'єднання використовуємо «Dbeaver».(див. рис. 5.2.4)



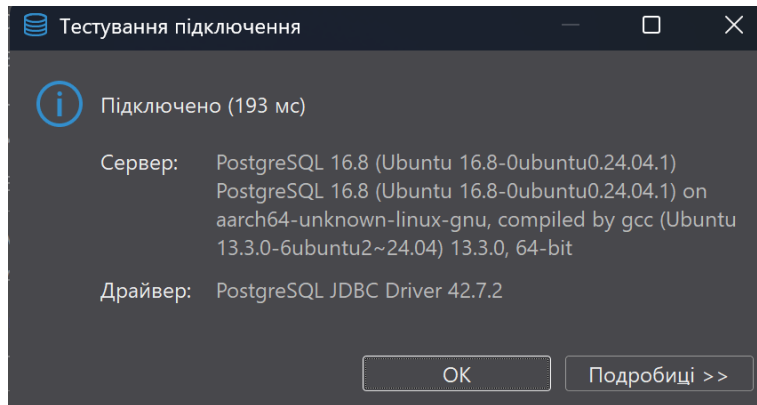


Рисунок 5.2.4 – Успішне під’єднання бази по мережі

Результатом даного пункту є підготовлений сервер дистанційного підключення. Даний пункт покрила задача kan-35. Було витрачено близько п’ять годин на виконання. Питання закрито в планових термінах.

6. Сайт

6.1. Проектування сайту

В цілях задовільнення вимог проекту, було вирішено створити односторінковий сайт з верхнім меню. Верхнє меню створене для управління аккаунтами користувачів, на рис. 6.1.1 показана діаграма запроектована на показ станів і дій через верхнє меню.

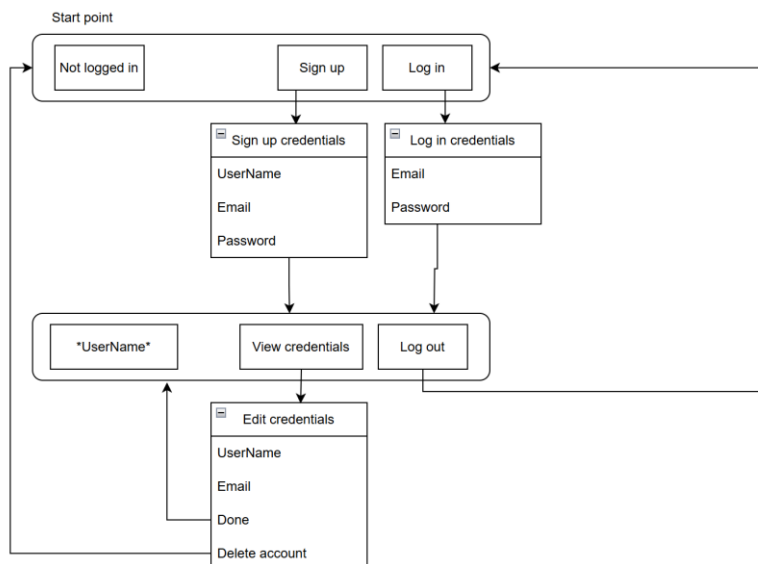


Рисунок 6.1.1 – Діаграма верхнього меню

Програмування сайту грубо поділяється на два етапи: “Backend” (“бекенд”) та “Frontend” (“фронтенд”). “Frontend” - це та частина веб-додатка, з якою взаємодіє користувач. Вона охоплює дизайн, інтерактивність і представлення

даних. “Backend” - це серверна частина веб-додатка, яка займається обробкою даних, взаємодією з базами даних і виконанням бізнес-логіки.

Для полегшення роботи над обома частинами було використано два фреймворки (“Framework”) –інфраструктури програмних рішень, що створені для розробки складних систем. Спрощено дану інфраструктуру можна вважати своєрідною комплексною бібліотекою, але при цьому вона має ряд обмежень, що задають правила створення структури проєкту та написання коду.

Для розробки “Backend” було обрано «Microsoft .NET core 9» з кількох особистих та глобальних переваг даного фреймворка:

- Основна мова роботи на фреймворці с#, але не обмежено можна користуватися і набором інших мов
- Крос-платформеність. Застосунок на фреймворці буде працювати на більшості операційних систем
- Велике ком’юніті і популярність, що допомагає швидкому навчанню
- Висока продуктивність та швидкість роботи
- Модульність, що допомагає використовувати те що потрібно і не перевантажувати програму
- Безкоштовна та легко керується з IDE(Інтегроване середовище розробки) Visual Studio

Для розробки “Frontend” було обрано «Vue» з кількох причин:

- Оригінально розрахований на створення односторінкових сайтів
- Легкість в вивченні через використання HTML, CSS та JavaScript в одному файлі та читабельності
- Не впливає на продуктивність роботи оскільки займає мінімум місця
- Легкість поєднання з іншими бібліотеками для розширення функціоналу
- Велике ком’юніті і популярність

А також даний фреймворк було створено на “Vite” оскільки він допомагає білду сайту завантажитись в секунди і практично миттєво оновлювати збережені зміни в реальному часі.

Раніше масово для виконання задачі збірки модулів, тобто правильного створення залежностей файлів з якими пізніше працює «index.html», використовували “Webpack”. Але його конкурент “Vite” зміг досягти швидшого виконання даної роботи за допомогою новіших технологій.

Результатом даного пункту є запроектований сайт. Даний пункт покрила задача kan-31. Було витрачено приблизно пів години на виконання. Питання закрито в планових термінах.

6.2. Реалізація бази даних – рівень data

Після створення бази даних необхідно задати їй таблиці з стовбцями, а також обмеження та з'єднання. Даний етап відбувається в «Microsoft .NET core 9» через модуль Data.

Перед початком потрібно додати "ConnectionString" щоб програма знала де знаходиться база даних для роботи. Тому в відповідному файлі appsettings.Development.json додаємо код:

```
"ConnectionStrings": {
  "web_kursachsem4.dev":
  "Host=192.168.1.62;Port=5432;Username=sonik;Password=post1238;Database=tetris;"
},
```

Для реалізації БД використовуємо “dotnet Entity Framework (EF) Core” — це сучасний об'єктно-реляційний відображувач (ORM) для .NET. Він дозволяє .NET працювати з базами даних, використовуючи об'єкти .NET, усуваючи необхідність у більшій частині коду доступу до даних, який зазвичай потрібно писати.

В модулі web_kursachsem4.Data.Models файлі models.cs створюємо класи, які слугуватимуть моделями таблиць в БД. Тут задаємо назви таблиць і колонок (частина файлу):

```
public class User
{
  // Первинний ключ
  public int UserId { get; set; }

  public string UserName { get; set; }
  public string Email { get; set; }
  public string Password { get; set; }

  // Навігація
  public virtual Score Score { get; set; }
}

public class Score
{
  public int UserId { get; set; } // PK та FK
  public string UserName { get; set; }
  public int ScoreValue { get; set; }
```

```

    public virtual User User { get; set; }
    public virtual User UserByName { get; set; }
}

```

Далі в файлі mainDBcontext.cs задаємо обмеження та з'єднання:

```

using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Text;
using System.Text.Json;
using web_kursachsem4.Data.Models;

namespace web_kursachsem4.Data
{
    public class mainDBcontext : DbContext
    {
        public mainDBcontext(DbContextOptions<mainDBcontext> options) : base(options)
        {
        }

        public virtual DbSet<User> Users { get; set; }
        public virtual DbSet<Score> Scores { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<User>(entity =>
            {
                entity.HasKey(u => u.UserId);

                entity.Property(u => u.UserName).IsRequired().HasMaxLength(100);
                entity.HasIndex(u => u.UserName).IsUnique();
                entity.HasIndex(u => u.Email).IsUnique();
                entity.Property(u => u.Email).IsRequired();
                entity.Property(u => u.Password).IsRequired();

                // User має один Score, Score має одного User, зовнішній ключ у Score
                entity.HasOne(u => u.Score)
                    .WithOne(s => s.User)
                    .HasForeignKey<Score>(s => s.UserId);
            });

            modelBuilder.Entity<Score>(entity =>
            {
                entity.HasKey(s => s.UserId);

                entity.Property(s => s.ScoreValue).IsRequired();
                entity.HasOne(s => s.UserByName)
                    .WithOne()
                    .HasPrincipalKey<User>(u => u.UserName)

```

```

        .HasForeignKey<Score>(s => s.UserName)
        .onDelete(DeleteBehavior.Restrict);
    });
}
}
}

```

Для перенесення написаного в БД необхідно створити міграцію і завантажити БД яку ми прописали в коді вище. Для цього використовуємо команди “dotnet ef”. Щоб не прописувати кожен раз довгі команди використовуємо можливості “Makefile” – способу автоматизувати виконання декількох чи об’ємних команд меншою командою.

Вміст Makefile:

```
#Project Variables
```

```
PROJECT_NAME ?= web_kursachsem4
```

```
.PHONY: migrations db hello
```

```
migrations:
```

```
    dotnet ef --startup-project .\web_kursachsem4.Web\web_kursachsem4.Web.csproj --project
.\web_kursachsem4.Data\web_kursachsem4.Data.csproj migrations add $(mname)
```

```
db:
```

```
    dotnet ef --startup-project .\web_kursachsem4.Web\web_kursachsem4.Web.csproj --project
.\web_kursachsem4.Data\web_kursachsem4.Data.csproj database update
```

```
hello:
```

```
    echo 'hello!'
```

Тепер для створення міграції пишемо: `make migrations mname=*name*`

А для апдейту БД: `make db`

Вигляд реалізованої діаграми БД показано на рис. 6.2.1.

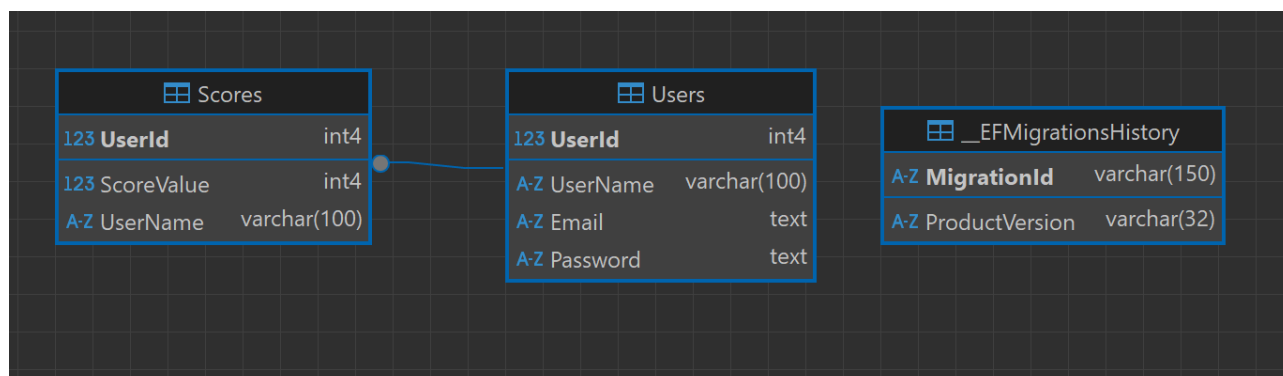
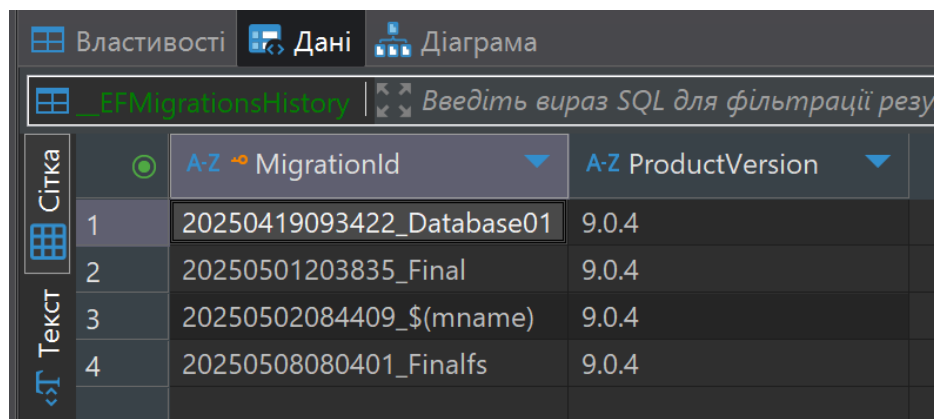


Рисунок 6.2.1 – Діаграма БД

Таблиця «__EFMigrationsHistory» створюється автоматично і вміщає в собі історію всіх міграцій.(див. рис. 6.2.2)



	MigrationId	ProductVersion
1	20250419093422_Database01	9.0.4
2	20250501203835_Final	9.0.4
3	20250502084409_\$(mname)	9.0.4
4	20250508080401_Finalfs	9.0.4

Рисунок 6.2.2 – Вміст таблиці «__EFMigrationsHistory»

Результатом даного пункту є реалізована БД. Даний пункт покрила задача кап-44. Було витрачено близько дві години на виконання. Питання закрито в планових термінах.

6.3. SQL запити – рівень service

Після реалізації бази даних необхідно створити запити до неї, які потребує логіка проекту. Даний етап відбувається в «Microsoft .NET core 9» через модуль Service.

В ньому налаштовуємо два файли: Exceptions.cs та MainService.cs.

Перший файл необхідний для виведення повідомлень в консоль. Вміст Exceptions.cs:

```
using System;
```

```
namespace web_kursachsem4.Exceptions
{
    public class NotFoundException : Exception
    {
        public NotFoundException() : base() { }
        public NotFoundException(string message) : base(message) { }
        public NotFoundException(string message, Exception innerException) : base(message,
innerException) { }
        public NotFoundException(string entityName, object key)
            : base($"Entity \"{entityName}\" ({key}) was not found.") { }
    }
}
```

Основна мета цього коду - створити спеціалізований тип винятку NotFoundException, який розширює стандартний клас Exception. Цей користувацький виняток призначений для використання у випадках, коли в

програмі не вдається знайти певний ресурс або сутність (наприклад, запис у базі даних, файл тощо).

Другий файл поділяється на інтерфейс і клас що його реалізує.

В інтерфейсі визначаємо асинхронні таски які або повертають результат або виконують дію з базою даних. Код інтерфейсу:

```
public interface IMainService
{
    Task<string?> GetUsernameAsync(int userId); // Отримання значення UserName за UserId
    Task<string?> GetEmailAsync(int userId); // Отримання значення Email за UserId
    Task<User> AddUserAsync(User user); // Створення структури User
    Task DeleteUserAsync(int userId); // Видалення структури User за UserId

    Task<int?> GetScoreAsync(int userId); // Отримання значення Score за UserId
    Task<EditScoreResult> EditScoreAsync(int userId, int score); // Заміна значення Score за UserId

    Task<Score[]> GetLeaderBoard(); // Отримання 10 відсортованих за значенням Score
    структур Score
    Task<User?> AuthenticateUserAsync(string username, string password); // Метод для
    автентифікації
}
```

Код до класу реалізації див. Додаток А.

Таск GetUsernameAsync для отримання імені користувача за його ID передбачає кейси:

- Користувача з вказаним ID не знайдено, в такому випадку повертається null
- Користувача з вказаним ID знайдено, в такому випадку повертається його ім'я (string).

Відбувається запит до бази даних `_db.Users` з фільтрацією за `UserId` та вибіркою поля `UserName` за допомогою методів LINQ `Where` та `Select`. Результат отримується асинхронно за допомогою `FirstOrDefaultAsync`.

Таск GetEmailAsync для отримання email користувача за його ID передбачає кейси:

- Користувача з вказаним ID не знайдено, в такому випадку повертається null.
- Користувача з вказаним ID знайдено, в такому випадку повертається його email (string).

Відбувається запит до бази даних `_db.Users` з фільтрацією за `UserId` та вибіркою поля `Email` за допомогою методів LINQ `Where` та `Select`. Результат отримується асинхронно за допомогою `FirstOrDefaultAsync`.

Таск DeleteUserAsync для видалення користувача за його ID передбачає кейси:

- Користувача з вказаним ID не знайдено в базі даних, в такому випадку викидається виняток `NotFoundException`.
- Успішне видалення користувача

Користувач видаляється з контексту бази даних `_db.Users` за допомогою `Remove()`. Зміни зберігаються в базі даних асинхронно за допомогою `SaveChangesAsync()`. Модель налаштована на каскадне видалення пов'язаних записів `Score`.

Task `AddUserAsync` для додавання нового користувача передбачаються кейси:

- Передано `null` в якості об'єкта `User`, в такому випадку викидається виняток `ArgumentNullException`
- Передано об'єкт `User` з пустим `UserName` або `Password`, в такому випадку викидається виняток `ArgumentException`.
- Користувач з таким `UserName` вже існує в базі даних, в такому випадку викидається виняток `InvalidOperationException`.
- Успішне додавання нового користувача

Пароль користувача хешується за допомогою `BCrypt.Net.BCrypt.HashPassword(user.Password)`. Користувач додається до контексту бази даних `_db.Users`. Зміни зберігаються в базі даних асинхронно за допомогою `SaveChangesAsync()`, після чого користувачу присвоюється ID. Створюється початковий запис `Score` для цього користувача зі значенням 0 та додається до контексту `_db.Scores`. Зміни зберігаються в базі даних асинхронно за допомогою `SaveChangesAsync()`. Повертається доданий об'єкт `User` з присвоєним ID.

Task `GetScoreAsync` для отримання рахунку користувача за його ID передбачає кейси:

- Запис про рахунок користувача з вказаним ID не знайдено, в такому випадку повертається `null (int?)`.
- Запис про рахунок користувача з вказаним ID знайдено, в такому випадку повертається значення його рахунку (`int`).

Відбувається запит до бази даних `_db.Scores` з фільтрацією за `UserId` та вибіркою поля `ScoreValue` за допомогою методів LINQ `Where` та `Select`. Результат отримується асинхронно за допомогою `FirstOrDefaultAsync`.

Task `EditScoreAsync` для редагування рахунку користувача за його ID передбачає кейси:

- Передано від'ємне значення рахунку, в такому випадку викидається виняток `ArgumentException`.
- користувача з вказаним ID не знайдено в таблиці `Users`, в такому випадку викидається виняток `NotFoundException`.

- Для користувача з вказаним ID не знайдено запис у таблиці Scores, в такому випадку створюється новий запис Score з вказаним рахунком. Повертається об'єкт EditScoreResult зі статусом UpdatedOrCreated = true.
- Для користувача з вказаним ID знайдено запис у таблиці Score. Якщо переданий рахунок більший за існуючий, рахунок оновлюється в базі даних. Повертається об'єкт EditScoreResult зі статусом UpdatedOrCreated = true та попереднім значенням рахунку. Якщо переданий рахунок не більший за існуючий, оновлення в базі даних не відбувається. Повертається об'єкт EditScoreResult зі статусом UpdatedOrCreated = false та поточним значенням рахунку.

Відбувається пошук запису Score за UserId за допомогою FindAsync(). Залежно від результату виконується оновлення або створення нового запису та збереження змін за допомогою SaveChangesAsync().

Task GetLeaderBoard при створенні користувача окрім запису переданих даних робить перевірку

Task AuthenticateUserAsync при автентифікації користувача передбачені кейси:

- Якщо передано пустий або такий, що складається лише з пробілів, username або password, метод повертає null
- Повертає null, якщо користувача з таким логіном не знайдено.
- Якщо паролі не співпадають, метод повертає null, вказуючи на невірний пароль.
- Якщо користувача знайдено, відбувається верифікація введеного password з хешованим паролем, що зберігається в базі даних (user.Password)

Автентифікація відбувається за допомогою BCrypt.Net.BCrypt.Verify(password, user.Password); - методу .NET core 9 з пакету BCrypt.Net-Next.

Результатом даного пункту є реалізовані запити до БД. Даний пункт покрила задача kan-44. Було витрачено приблизно дві години на виконання. Питання закрито в планових термінах.

6.4. HTTP запити – рівень web

Після реалізації запитів бази даних необхідно створити HTTP запити. Даний етап відбувається в «Microsoft .NET core 9» через модуль Web.

Даний етап виконаний в основному файлі MainController.cs, але також має допоміжні файли в папці RequestModels. Допоміжні файли це можливі моделі запитів які може отримати та передати HTTP код. А саме:

Таблиця 6.4.1 – Вміст папки RequestModels

Назва запиту	Призначення
LoginRequest	Запит з Username та Password для авторизації
NewUserRequest	Запит з Username, Email та Password для створення користувача
ScoreChangeRequest	Запит з Score для зміни рахунку
TokenResponse	Відповідь з Token при аутентифікації

В файлі MainController.cs окрім методів реалізації HTTP запитів є декілька допоміжних методів: GenerateJwtToken та GetCurrentUserId. Обидва призначені для роботи з авторизацією користувача.

Метод GenerateJwtToken – метод що генерує JWT(“JSON Web Token” - це стандарт токена доступу на основі JSON), який діє дві години.

Метод GetCurrentUserId - метод що дістає ід користувача з JWT.

Через систему автентифікації є запити, які є недосяжними без JWT. Також кожен запит має відповідний метод на рівні Service для запиту на базу даних. Детальніше див. табл. 6.4.2

Таблиця 6.4.2 – Опис методів

Назва	Назва методу з Service	Необхідність авторизації	Призначення
Login	AuthenticateUserAsync	Відсутня	Автентифікація та авторизація користувача
AddUser	AddUserAsync	Відсутня	Створення нового користувача
GetMyUsername	GetUsernameAsync	Необхідна	Отримання ім'я поточного користувача з токена
GetMyEmail	GetEmailAsync	Необхідна	Отримання емейлу поточного користувача з токена
DeleteMyAccount	DeleteUserAsync	Необхідна	Видалення облікового запису поточного користувача
EditScore	EditScoreAsync	Необхідна	Оновлення рахунку поточного користувача
GetMyScore	GetScoreAsync	Необхідна	Отримання рахунку поточного користувача
GetLeaderboard	GetLeaderBoard	Відсутня	Отримання топ-10 рахунків
GetUsername	GetUsernameAsync	Необхідна	Отримання ім'я користувача за ід
GetTest	--	Відсутня	Швидка перевірки роботи системи

Кожен метод має обробку можливих помилок та відповідає статус кодами відповідними до успішності виконання запиту. Для перегляду коду див. Додаток Б.

Результатом даного пункту є реалізовані HTTP запити. Даний пункт покрила задача кап-44. Було витрачено приблизно чотири години на виконання. Питання закрито в планових термінах.

6.5. Фронтенд

Даний пункт виконував інший учасник – Оніщенко Є.О. Для деталізації див. звіт «Курсова_робота_ІО-32_Оніщенко.docx».

6.6. Передача даних з гри

Даний пункт виконував інший учасник – Оніщенко Є.О. Для деталізації див. звіт «Курсова_робота_ІО-32_Оніщенко.docx».

6.7. Взаємодія гри із сайтом

Даний пункт виконував інший учасник – Оніщенко Є.О. Для деталізації див. звіт «Курсова_робота_ІО-32_Оніщенко.docx».

7. Тестування

7.1. Модульне тестування

При розробці проекту відбувалось модульне тестування за допомогою тест кейсів.

Модульне тестування (Unit testing)– тестування кожної атомарної функціональності додатку окремо, в штучно створеному середовищі.

Тест-кейс — це набір вхідних значень, попередніх умов виконання, очікуваних результатів і післяумов, розроблених для конкретної мети або умови тестування.

Тестування проекту відбувалося за участі Оніщенко Є. О. та Куліченко С. Р.

Тестування відбулося в даних модулях:

- 7.1.1 Тестування підключення до БД
- 7.1.2 Тестування розробки гри на першому етапі
- 7.1.3 Тестування розробки гри на другому етапі
- 7.1.4 Тестування розробки гри на третьому етапі
- 7.1.5 Тестування HTTP запитів

7.1.1 Тестування підключення до БД

Номер	Назва	№	Кроки	Результат	Статус
7.1.1	Тестування підключення до БД	1	Відкриваємо “Dbeaver” та вводимо дані підключення, натискаємо перевірити з’єднання	Повинно відобразитись, що з’єднання успішне	Успіх

7.1.2 Тестування розробки гри на першому етапі

Номер	Назва	№	Кроки	Результат	Статус
7.1.2	Тестування розробки гри на першому етапі	1	Відкриваємо проект в “Unity” та запускаємо ігровий режим	Гра симулюється в ігровому просторі	Успіх
		2	Чекаємо генерації блока	Блок генерується і починає падати на панелі	Успіх
		3	Клацаємо стрілочки на клавіатурі(вліво, вправо)	Блок рухається в відповідно заданому стрілкою русі	Успіх
		4	Клацаємо клавіші “A” або “D” на клавіатурі	Блок рухається в ліво при нажатій “A”, в право при “D”	Успіх
		5	Клацаємо клавіші “W” або “Стрілка вгору”	Блок обертається	Успіх
		6	Клацаємо клавіші “S” або “Стрілка вниз”	Блок падає швидше	Успіх
		7	В безкінечному режимі складаємо блоки в ряд від стінки до стінки	Ряд пропадає, блоки що стояли вище падають на низ. Рахунок зростає	Успіх
		8	В рівневому режимі клацаємо клавіші “Q” та “E” або “Z” та “X”	На списку блоків на екрані: При “Q” або “Z” вибір пересувається вліво При “E” або “X” вибір пересувається вправо	Успіх

		9	В рівневому режимі клацаємо клавішу “Space”	Обраний блок з списку ставиться на початок і починає падати вниз	Успіх
		10	Встановлення блоку класичного	Блок стає займає пусте місце на сітці	Успіх
		11	Встановлення блоку легкого	Блок стає займає пусте місце на сітці або накладається на інший легкий блок	Успіх
		12	Встановлення блоку різниці	Блок стає займає пусте місце на сітці або накладається на інший легкий блок видаляючи його	Успіх
		13	Встановлення блоку пересікання	Блок стає займає пусте місце на сітці або накладається на інший легкий блок видаляючи його. Всі клітинки що залишились від нього вважаються легким блоком	Успіх

При тестуванні в кроці 12 була виявлена проблема, яку було усунуто в KAN-33.

7.1.3 Тестування розробки гри на другому етапі

Номер	Назва	№	Кроки	Результат	Статус
7.1.3	Тестування розробки гри на другому етапі	1	Відкриваємо проект в “Unity” та запускаємо гру	Гра починає симулювання в сцені меню	Успіх
		2	На екрані клацаємо курсором на кнопку “Endless mode”	Вмикається сцена з безкінечним ігровим режимом	Успіх
		3	На екрані клацаємо курсором на кнопки рівнів “1”, “2”, “3” ... “20”	Вмикається сцена рівневого режиму з відповідним рівнем	Успіх
		4	В рівневому режимі клацаємо клавішу “R”	Сцена з рівнем перезапускається	Успіх

		5	В рівневому режимі клацаємо клавішу “Esc”	Відкривається сцена меню	Успіх
		6	В безкінечному режимі клацаємо клавішу “Esc”	Відкривається сцена меню	Успіх
		7	В рівневому режимі клацаємо на знак питання на екрані	Висвічується підказка	Успіх

При тестуванні в кроці 1 була виявлена проблема, яку було усунуто в KAN-49.

7.1.4 Тестування розробки гри на третьому етапі

Номер	Назва	№	Кроки	Результат	Статус
7.1.4	Тестування розробки гри на третьому етапі	1	Відкриваємо проект в “Unity” та запускаємо гру	Гра починає симулювання в сцені меню і відіграє мелодія головного меню	Успіх
		2	На екрані натискаємо курсором на кнопку одного із рівнів або “Endless mode”	Відіграє звук натискання кнопки	Успіх
		3	Переходим у режим гри	Відіграє фонову мелодія	Успіх
		4	В режимі гри натискаємо клавішу “R”	Відіграє звук рестарту	Успіх
		5	В рівневому режимі натискаємо клавішу “Q” або “E”	Відіграє звук зміни блоку	Успіх
		6	В режимі гри встановлюємо блок	Відіграє звук встановлення блоку	Успіх
		7	В режимі гри повертаємо блок	Відіграє звук повороту блоку	Успіх
		8	В режимі безкінечної гри видаляємо рядок	Відіграє звук видалення рядка	Успіх
		9	В режимі безкінечної гри піднімаємо рівень	Відіграє звук підняття рівня	Успіх

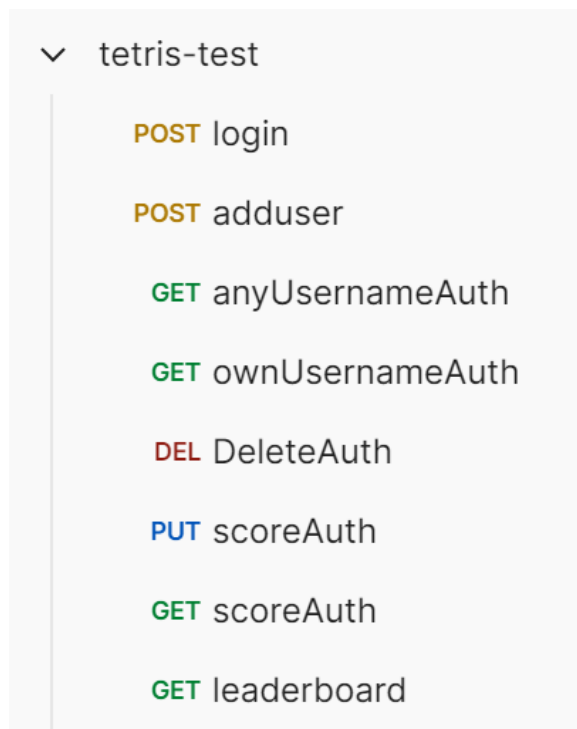
	10	В рівневому режимі проходимо рівень	Відіграє звук перемоги	Успіх
	11	В рівневому режимі не проходимо рівень	Відіграє звук поразки	Успіх
	12	В режимі безкінечної гри доходимо до вершини сцени	Відіграє звук поразки	Успіх

7.1.5 Тестування HTTP запитів

Номер	Назва	№	Кроки	Результат	Статус
7.1.5	Тестування HTTP запитів	1	Запит Post https://localhost:7167/api/auth/login	Повертає токен	Успіх
		2	Запит Post https://localhost:7167/api/users	Повертає створені структури User і Score	Успіх
		3	Запит Get https://localhost:7167/api/users/9/username	Повертає Username за UserId	Успіх
		4	Запит Get https://localhost:7167/api/me/username	Повертає Username за UserId в токені	Успіх
		5	Запит Delete https://localhost:7167/api/me	Видаляє записи за UserId в токені	Успіх
		6	Запит Get https://localhost:7167/api/me/score	Повертає Score за UserId в токені	Успіх
		7	Запит Get https://localhost:7167/api/leaderboard	Повертає 10 структур Score за UserId в токені	Успіх
		8	Запит Put https://localhost:7167/api/me/score	Замінює значення Score	Успіх

Для тестування відповідей від запитів було використано програму «Postman».
(див. рис. 7.1.5)

Рисунок 7.1.5 – Список запитів в програмі «Postman»



7.2 Тестування готового проекту для перевірки працездатності системи

Для тестування готового проекту перевірялись реалізації всіх прецедентів:

- Прецедент початкового запуску проекту.

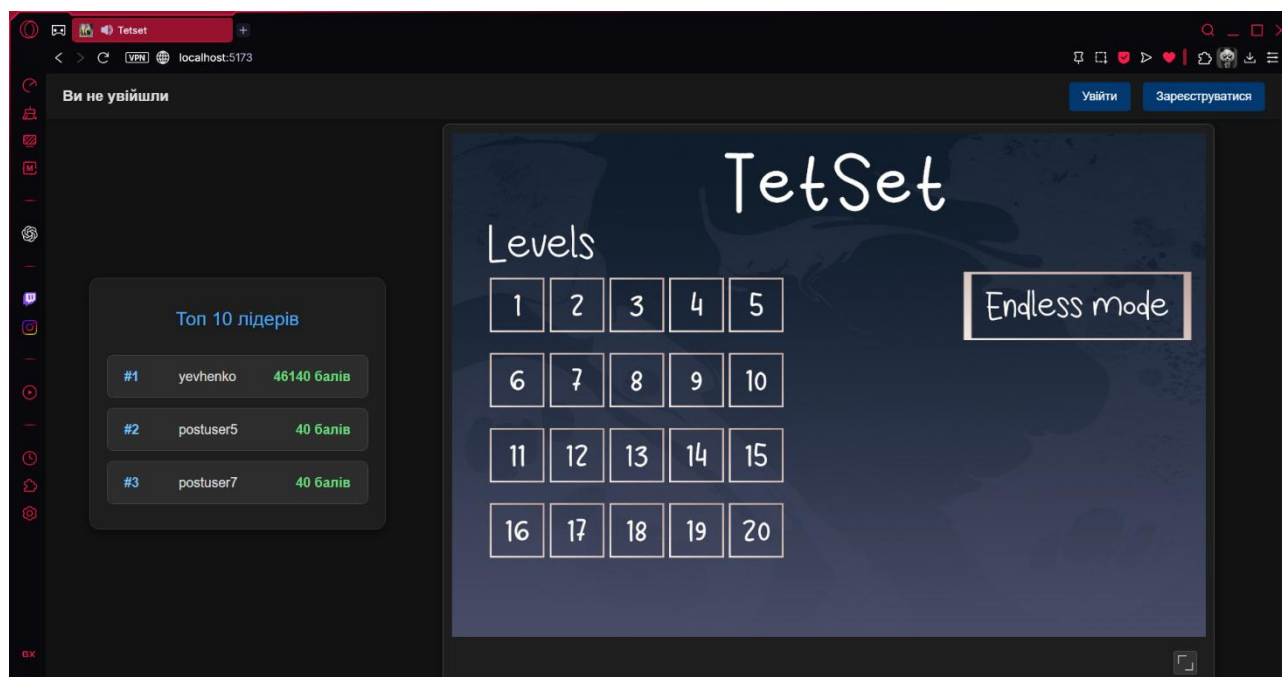


Рисунок 7.2.1

- Прецедент реєстрації нового користувача.

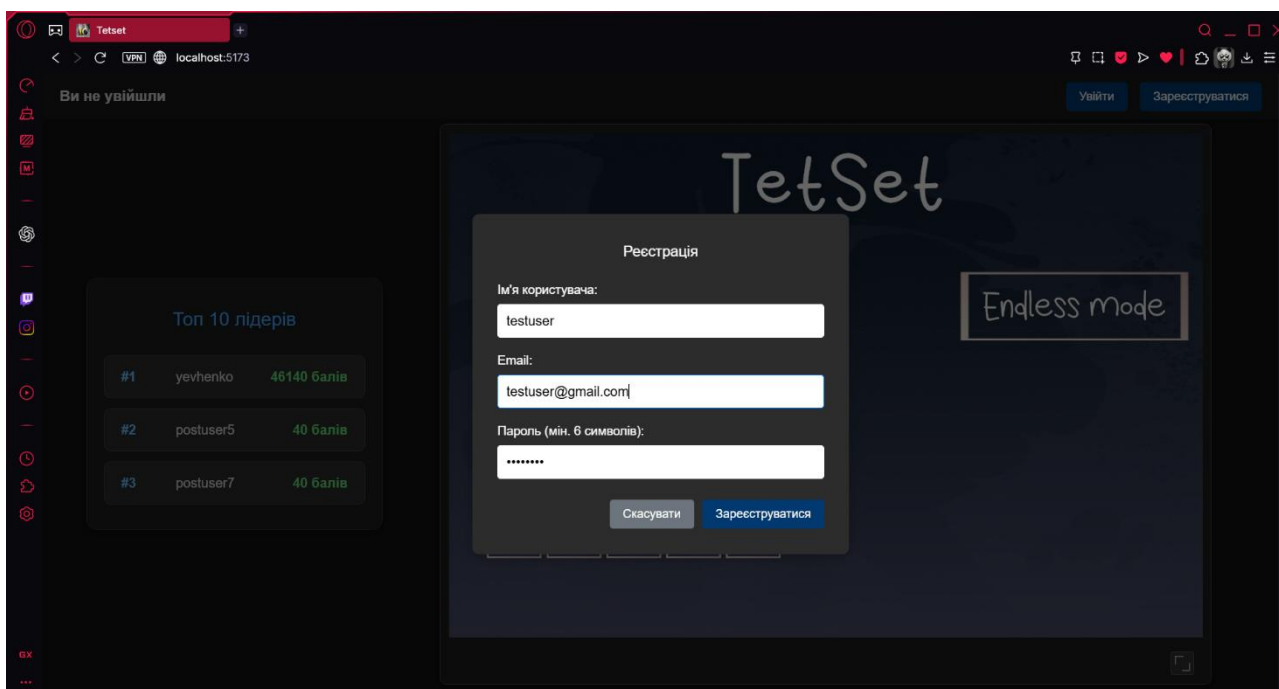


Рисунок 7.2.2

- Прецедент входу до акаунту.

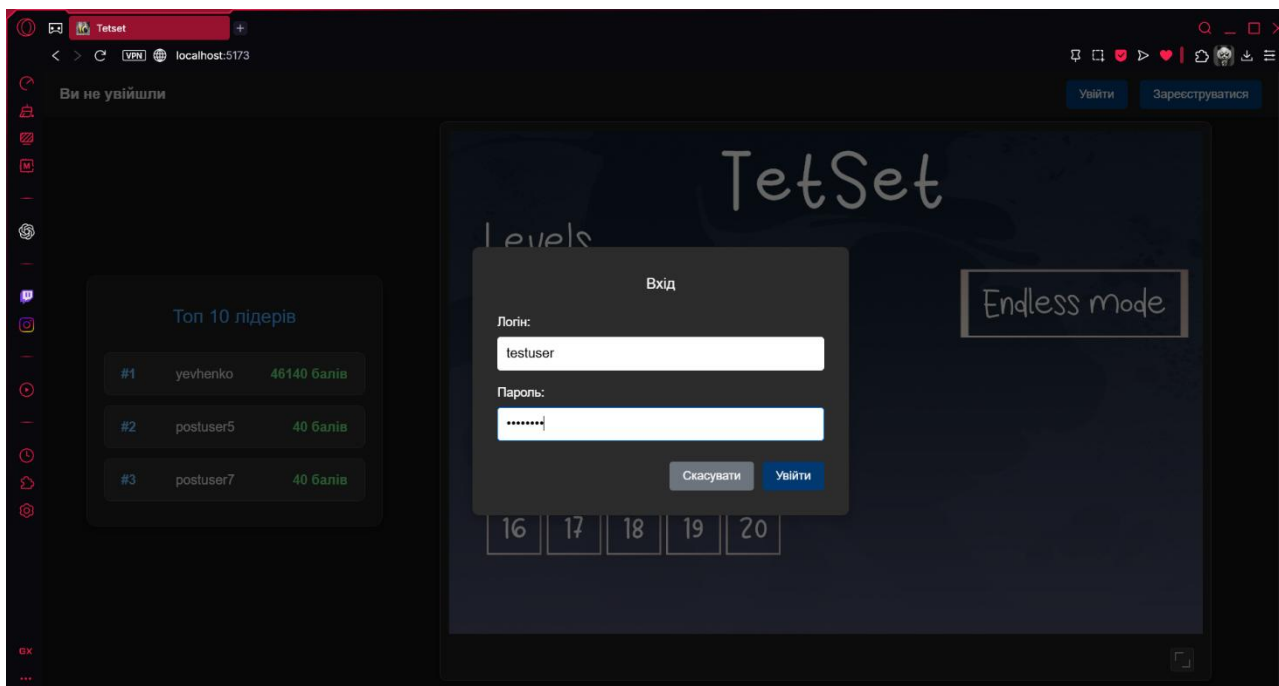


Рисунок 7.2.3

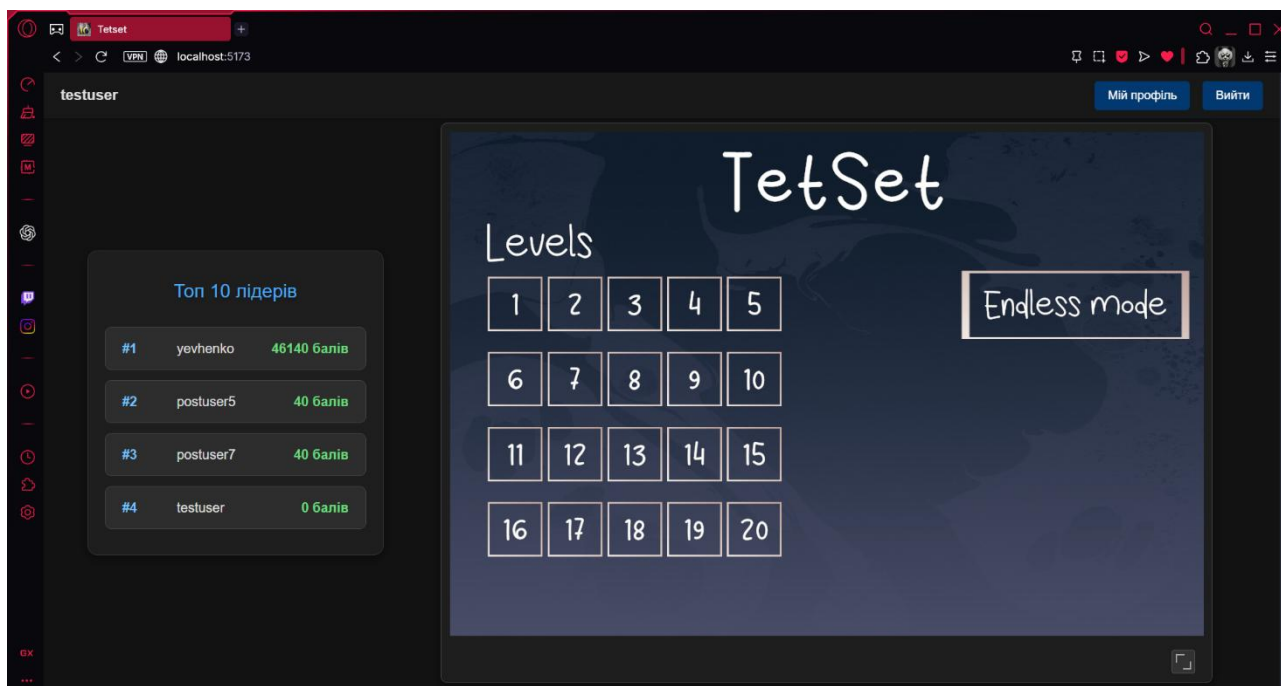


Рисунок 7.2.4

- Прецедент розкриття гри на весь екран.



Рисунок 7.2.5

- Прецедент запуску безкінечного режиму гри із ідентифікацією.

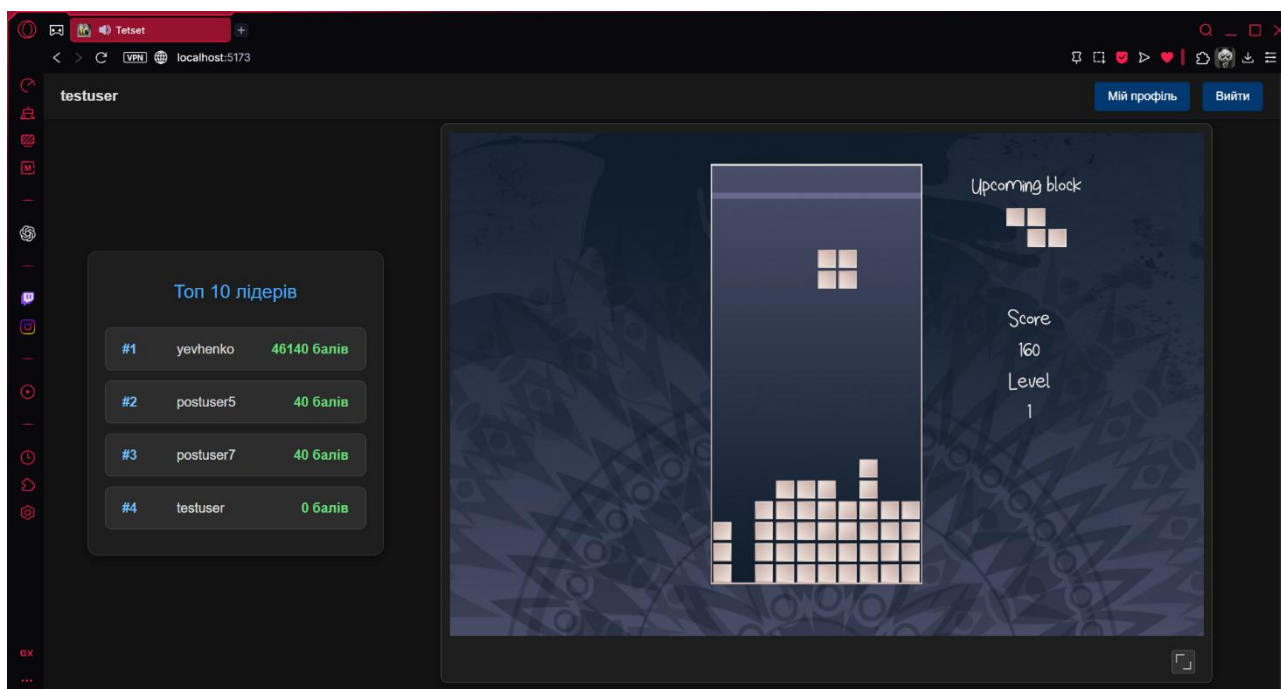


Рисунок 7.2.6

- Прецедент збереження результату безкінечного режиму гри. Як можна побачити, рахунок із рисунка 6.2.6 успішно зберігся до бази даних і відобразився у списку на сайті.

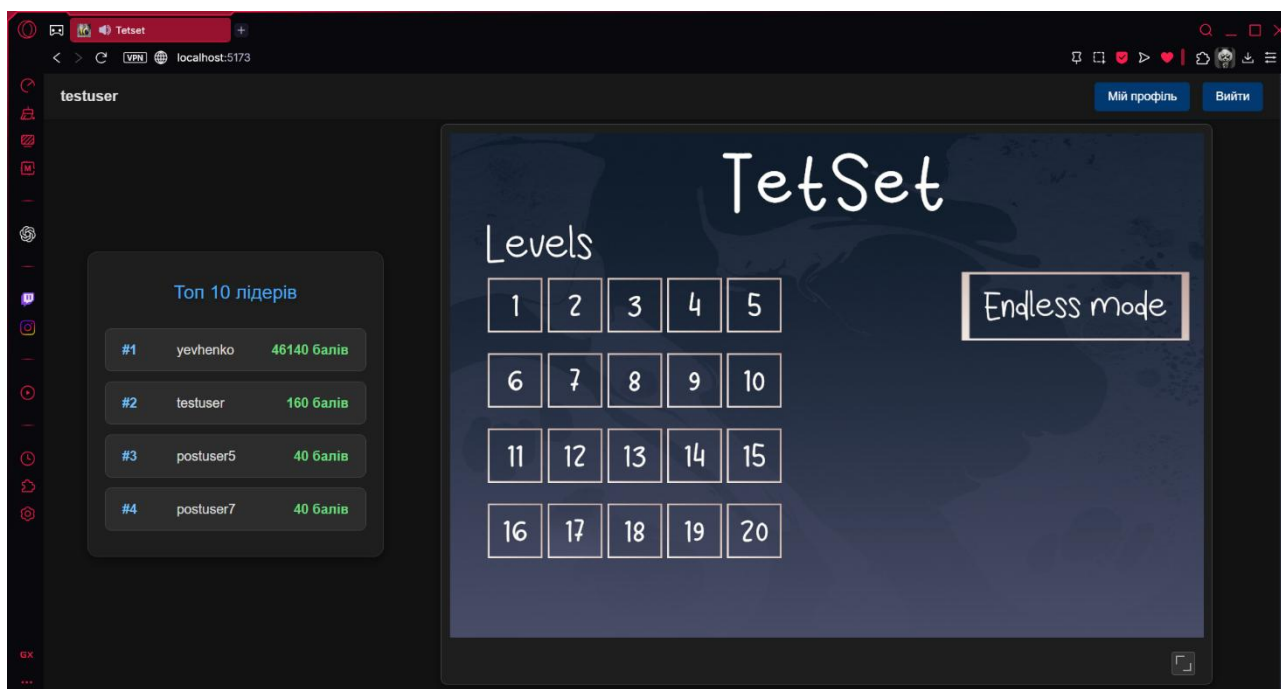


Рисунок 7.2.7

- Прецедент запуску безкінечного режиму гри без ідентифікації.

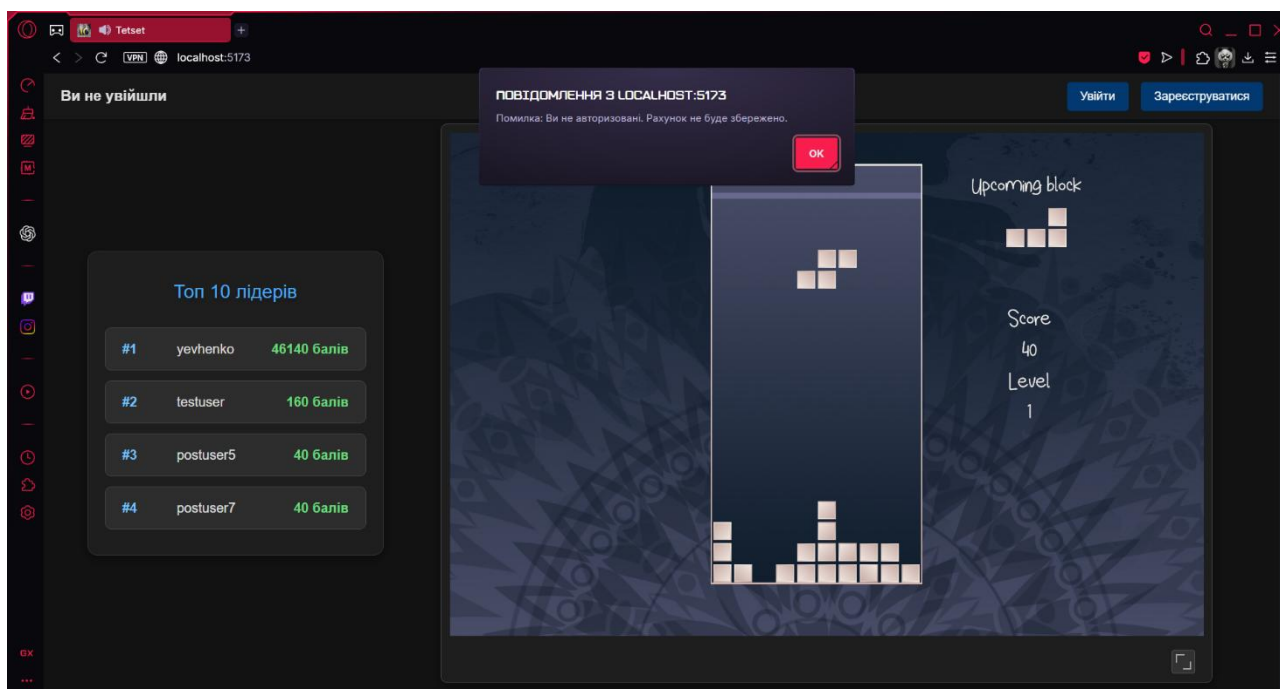


Рисунок 7.2.8

- Прецедент запуску одного із рівнів режиму множин.

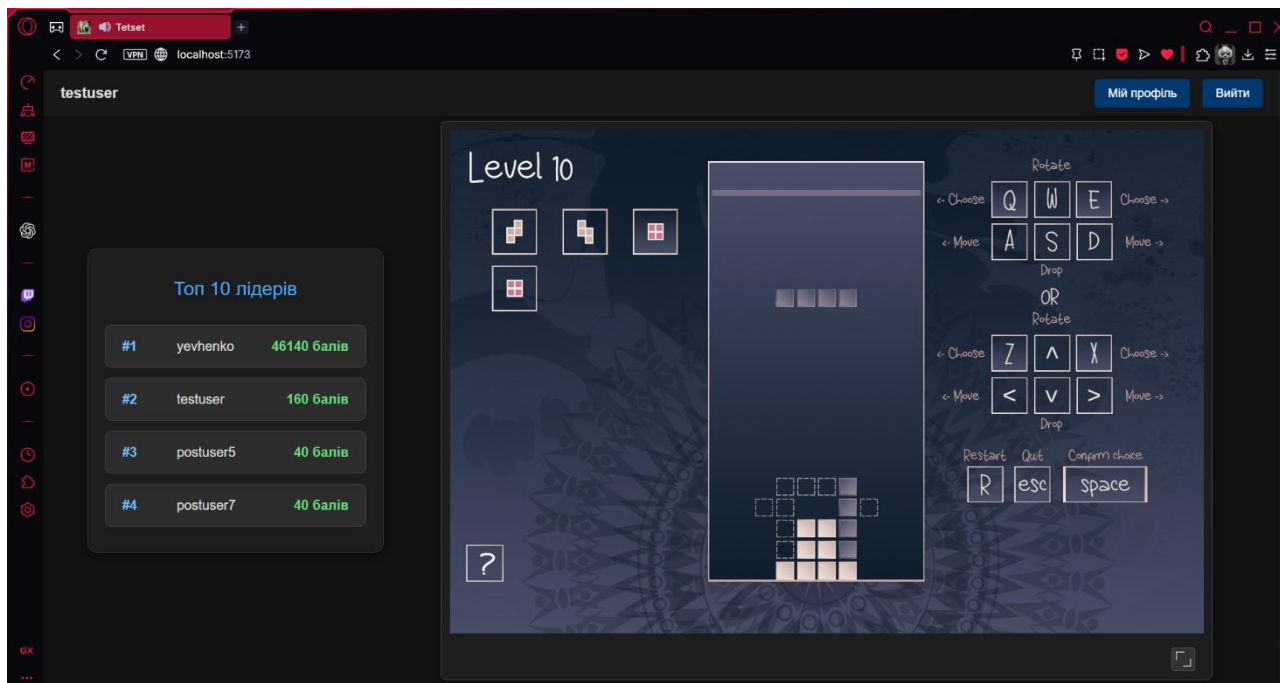


Рисунок 7.2.9

- Прецедент перемоги у режимі множин.

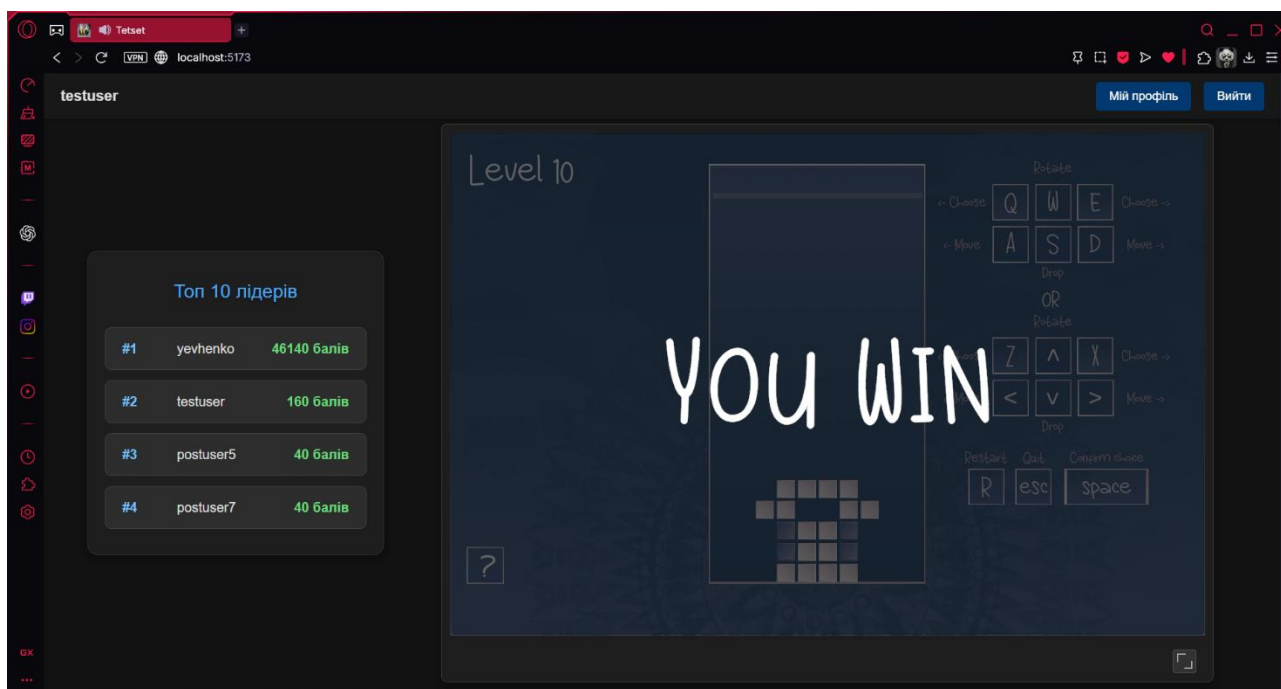


Рисунок 7.2.10

- Прецедент поразки у режимі множин.

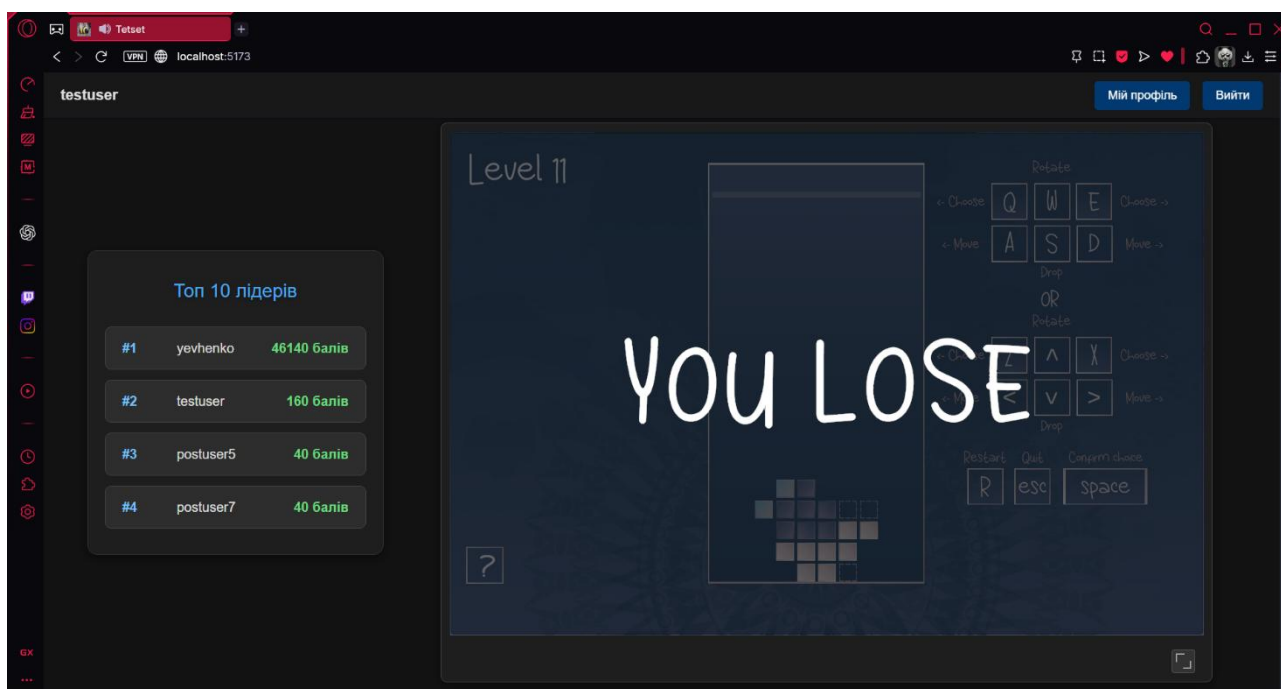


Рисунок 7.2.11

- Прецедент відкриття профілю користувача.

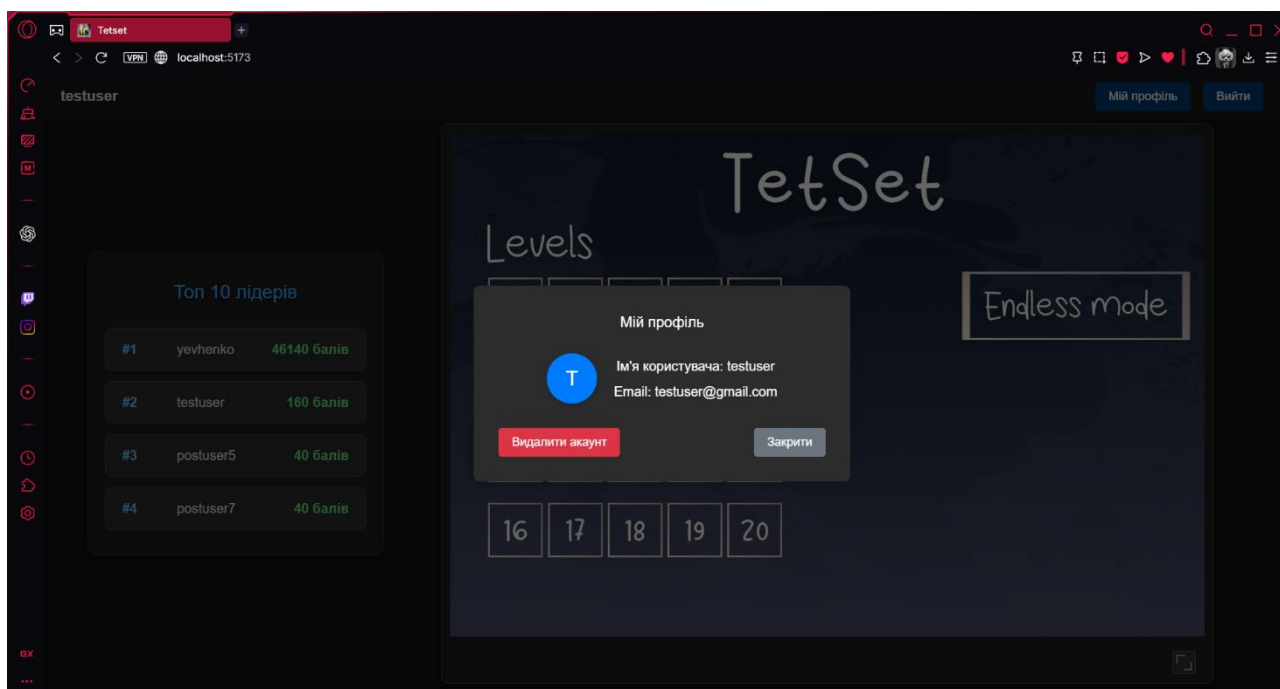


Рисунок 7.2.12

- Прецедент видалення профілю користувача.

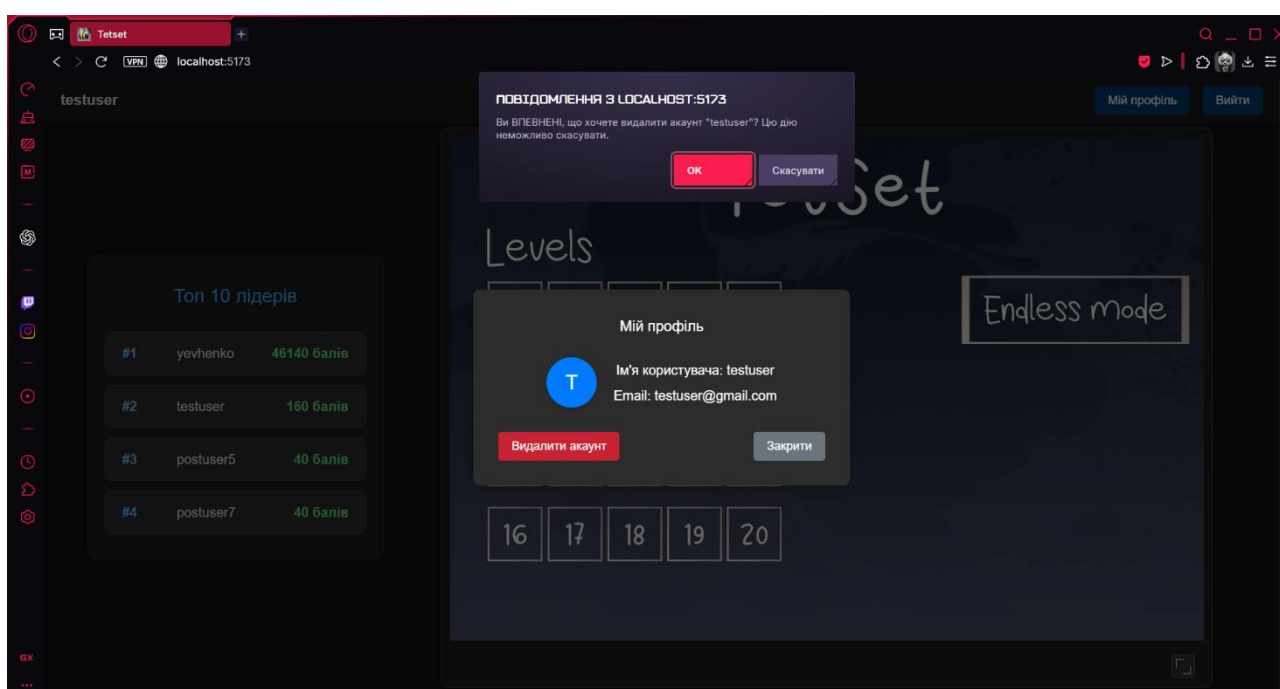


Рисунок 7.2.12

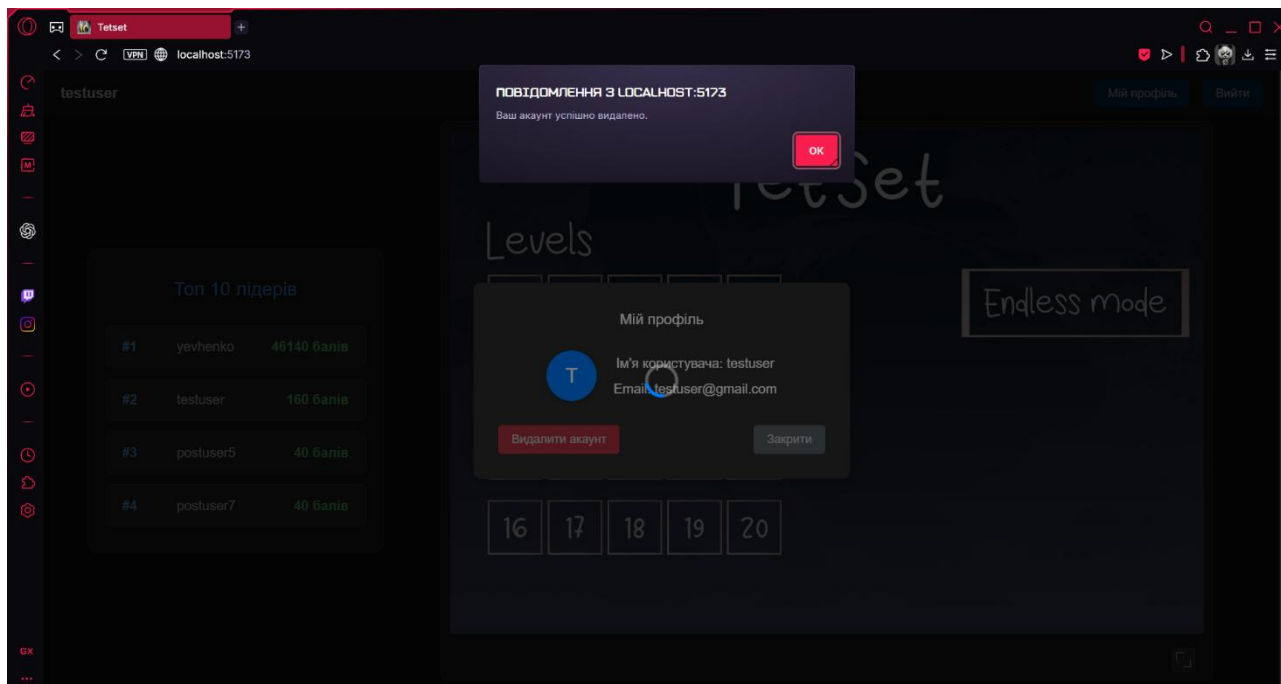


Рисунок 7.2.13

- Прецедент роботи серверу бекенду.

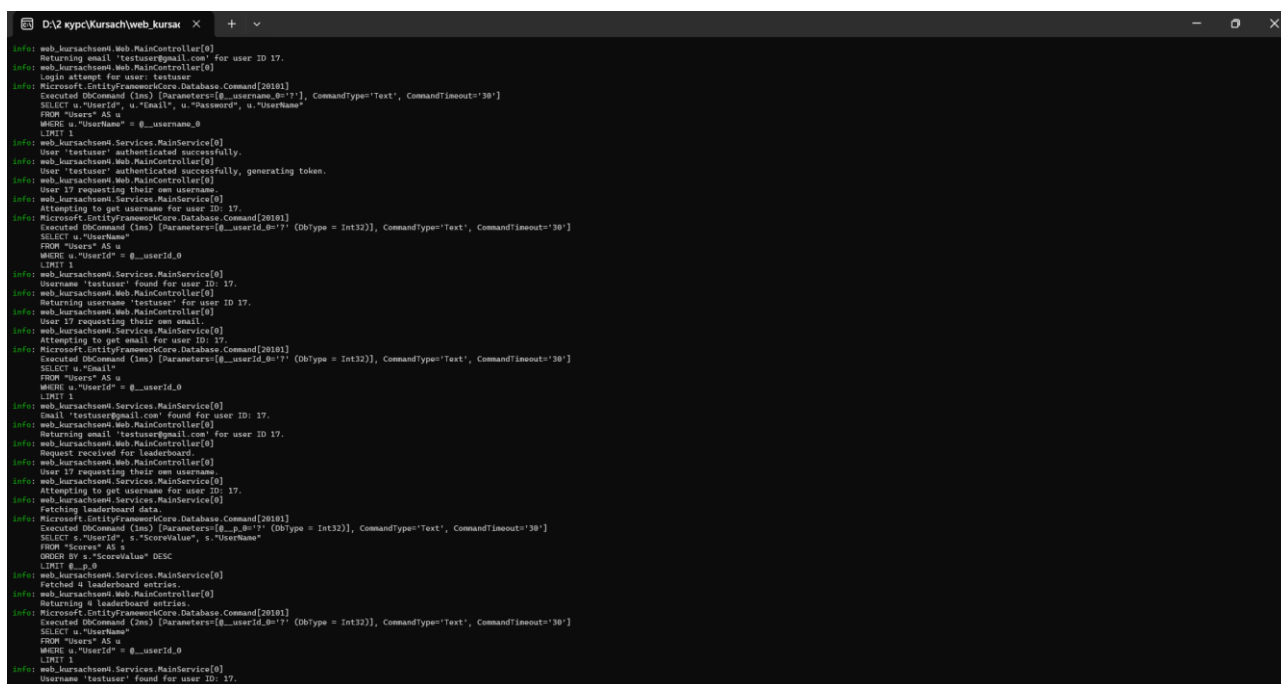


Рисунок 7.2.14

```

D:\2 kypc\Kursach\web_kursak x + v
info: web.kursachsm.Web.MainController[0]
Returning username 'testuser' for user ID 17.
info: web.kursachsm.Web.MainController[0]
User 17 requesting their own email.
info: web.kursachsm.Services.MainService[0]
Attempting to get email for user ID: 17.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (1ms) [Parameters=[@_userId=17] (DbType = Int32)], CommandType='Text', CommandTimeout='30'
SELECT u."Email"
FROM "Users" AS u
WHERE u."UserId" = @_userId_0
LIMIT 1
info: web.kursachsm.Services.MainService[0]
Email 'testuser@gmail.com' found for user ID: 17.
info: web.kursachsm.Web.MainController[0]
Returning email 'testuser@gmail.com' for user ID 17.
info: web.kursachsm.Web.MainController[0]
User 17 attempting to edit their score with value: 160.
info: web.kursachsm.Services.MainService[0]
Attempting to edit score for user ID: 17 with value: 160.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (3ms) [Parameters=[@_p_0=17] (DbType = Int32)], CommandType='Text', CommandTimeout='30'
SELECT s."UserId", s."ScoreValue", s."UserName"
FROM "Scores" AS s
WHERE s."UserId" = @_p_0
LIMIT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (6ms) [Parameters=[@p1=17] (DbType = Int32), @p0=17] (DbType = Int32)], CommandType='Text', CommandTimeout='30'
UPDATE "Scores" SET "ScoreValue" = @p0
WHERE "UserId" = @p1
info: web.kursachsm.Services.MainService[0]
Score updated for user ID: 17 from 0 to 160.
info: web.kursachsm.Web.MainController[0]
EditScore result for user 17: UpdatedOrCreated=True, FinalScore=160, Message="Пынешь ycnTamo omonemo."
info: web.kursachsm.Web.MainController[0]
Request received for leaderboard.
info: web.kursachsm.Services.MainService[0]
Fetching leaderboard data.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (1ms) [Parameters=[@_p_0=17] (DbType = Int32)], CommandType='Text', CommandTimeout='30'
SELECT s."UserId", s."ScoreValue", s."UserName"
FROM "Scores" AS s
ORDER BY s."ScoreValue" DESC
LIMIT @__p_0
info: web.kursachsm.Services.MainService[0]
Fetched 4 leaderboard entries.
info: web.kursachsm.Web.MainController[0]
Returning 4 leaderboard entries.
info: web.kursachsm.Web.MainController[0]
User 17 attempting to delete their own account.
info: web.kursachsm.Services.MainService[0]
Attempting to delete user with ID: 17.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (1ms) [Parameters=[@_p_0=17] (DbType = Int32)], CommandType='Text', CommandTimeout='30'
SELECT u."UserId", u."Email", u."Password", u."UserName"
FROM "Users" AS u
WHERE u."UserId" = @_p_0
LIMIT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (0ms) [Parameters=[@p0=17] (DbType = Int32)], CommandType='Text', CommandTimeout='30'
DELETE FROM "Users"
WHERE "UserId" = @p0
info: web.kursachsm.Services.MainService[0]
User with ID 17 deleted successfully.
info: web.kursachsm.Web.MainController[0]
User 17 deleted successfully.

```

Рисунок 7.2.15

Висновок

При виконанні курсової роботи були використанні знання, отримані при вивченні курсу «Інженерія програмного забезпечення» та «Організація баз даних», і які були підкріплені практичною роботою над проектуванням, розробкою та тестуванням проекту.

Результатом курсової роботи є реалізація гри за допомогою ігрового рушія «Unity» та подальша її публікація за допомогою локального хостингу на сайті створеного на фреймворці .NET core 9 та Vue+Vite. А також створення та менеджмент бази даних PostgreSQL на одноплатному комп'ютері Raspberry Pi 3b.

В результаті, всі функціональні і не функціональні вимоги, які були поставлені на початку і оголошені у технічному завданні були досягнуті.

Джерела

- Документація .Net Core: <https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-9.0>
- Форум по .Net Core: <https://techcommunity.microsoft.com/category/dotnet>
- Документація Unity: <https://docs.unity.com>
- Форум по Unity: <https://discussions.unity.com>
- Форум Stack Overflow: <https://stackoverflow.co>
- ТUTORIAL з YouTube: https://www.youtube.com/playlist?list=PL3_YUnRN3Uhi6WH0owikCdLqIK4l60EiT
- ТUTORIAL з YouTube: <https://www.youtube.com/watch?v=T5P8ohdxDjo&t=652s>

Додаток А

```

using System;
using web_kursachsem4.Data.Models;
using web_kursachsem4.Data;
using System.Linq;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;
using System.Threading.Tasks;
using web_kursachsem4.Exceptions;
using BCrypt.Net;
using Microsoft.Extensions.Logging;

namespace web_kursachsem4.Services
{
    // Інтерфейс з асинхронними методами
    public interface IMainService
    {
        Task<string?> GetUsernameAsync(int userId); // Може повернути null, якщо
не знайдено
        Task<string?> GetEmailAsync(int userId);
        Task<User> AddUserAsync(User user); // Повертаємо доданого користувача
(з ID)
        Task DeleteUserAsync(int userId);

        Task<int?> GetScoreAsync(int userId); // Може повернути null
        // Змінюємо сигнатуру: тепер повертаємо Task<EditScoreResult>
        Task<EditScoreResult> EditScoreAsync(int userId, int score);

        Task<Score[]> GetLeaderBoard();
        Task<User?> AuthenticateUserAsync(string username, string password); //
Повертає User при успіху, null при невдачі
    }

    // Реалізація сервісу з async/await та обробкою помилок
    public class MainService : IMainService
    {
        private readonly mainDBcontext _db;
        private readonly ILogger<MainService> _logger; // Додаємо логер

        // Оновлюємо primary constructor для інжекції логера
        public MainService(mainDBcontext db, ILogger<MainService> logger)
        {

```

```

        _db = db ?? throw new ArgumentNullException(nameof(db));
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }

    // --- User Methods ---
    public async Task<User?> AuthenticateUserAsync(string username, string
password)
    {
        if (string.IsNullOrEmpty(username) ||
string.IsNullOrEmpty(password))
        {
            _logger.LogWarning("AuthenticateUserAsync called with empty username
or password.");
            return null; // Неправильні вхідні дані
        }

        // Знаходимо користувача за ім'ям (регістрозалежне порівняння за
замовчуванням)
        var user = await _db.Users.FirstOrDefaultAsync(u => u.UserName ==
username);

        if (user == null)
        {
            _logger.LogInformation("Authentication failed: User '{Username}' not
found.", username);
            return null;
        }

        // user.Password - це збережений хеш з бази даних
        // password - це простий пароль, введений користувачем під час логіну
        bool isValidPassword = BCrypt.Net.BCrypt.Verify(password,
user.Password);

        if (isValidPassword)
        {
            _logger.LogInformation("User '{Username}' authenticated successfully.",
username);
            return user;
        }
        else
        {
            _logger.LogInformation("Authentication failed: Invalid password for user
'{Username}'.", username);
            return null;
        }
    }
}

```

```

public async Task<User> AddUserAsync(User user)
{
    if (user == null)
    {
        throw new ArgumentNullException(nameof(user));
    }
    if (string.IsNullOrEmpty(user.UserName) ||
string.IsNullOrEmpty(user.Password))
    {
        throw new ArgumentException("Username and password cannot be
empty.", nameof(user));
    }

    // перевірка на унікальність UserName перед додаванням
    if (await _db.Users.AnyAsync(u => u.UserName == user.UserName))
    {
        _logger.LogWarning("Attempted to add user with existing username:
'{Username}'.", user.UserName);
        // Краще кидати більш специфічний виняток, наприклад,
CustomValidationException або повернути статус помилки
        // Але InvalidOperationException теж робочий варіант.
        throw new InvalidOperationException($"User with username
'{user.UserName}' already exists.");
    }

    // --- Хешування Пароля ---
    user.Password = BCrypt.Net.BCrypt.HashPassword(user.Password);

    _db.Users.Add(user);
    await _db.SaveChangesAsync(); // Зберігаємо користувача, EF Core
присвоїть ID

    // Тепер user.UserId має значення, можна створювати залежні записи
    var initialScore = new Score { UserId = user.UserId, UserName =
user.UserName, ScoreValue = 0 };

    _db.Scores.Add(initialScore);

    await _db.SaveChangesAsync(); // Зберігаємо залежні записи

    _logger.LogInformation("User '{Username}' (ID: {UserId}) and initial score
created successfully.", user.UserName, user.UserId);

```

```

        return user; // Повертаємо доданого користувача
    }

    public async Task DeleteUserAsync(int userId)
    {
        _logger.LogInformation("Attempting to delete user with ID: {UserId}.",
userId);
        var userToDelete = await _db.Users.FindAsync(userId);

        if (userToDelete == null)
        {
            _logger.LogWarning("Deletion failed: User with ID {UserId} not found.",
userId);
            throw new NotFoundException(nameof(User), userId);
        }

        // каскадне видалення налаштоване в моделі/контексті БД для зв'язку
        User -> Score,

        _db.Users.Remove(userToDelete); // Видаляємо користувача
        await _db.SaveChangesAsync(); // Зберігаємо зміни (включно з
каскадними, якщо налаштовані)
        _logger.LogInformation("User with ID {UserId} deleted successfully.",
userId);
    }

    // Оптимізовано: вибираємо тільки UserName
    public async Task<string?> GetUsernameAsync(int userId)
    {
        _logger.LogInformation("Attempting to get username for user ID: {UserId}.",
userId);
        // Використовуйте Select та FirstOrDefaultAsync для отримання тільки
потрібного поля
        var username = await _db.Users
            .Where(u => u.UserId == userId)
            .Select(u => u.UserName)
            .FirstOrDefaultAsync();

        if (username == null)
        {
            _logger.LogInformation("Username not found for user ID: {UserId}.",
userId);
        }
        else
        {

```



```

        _logger.LogInformation("Username '{Username}' found for user ID:
{UserId}.", username, userId);
    }

    // FirstOrDefaultAsync поверне null, якщо користувача не знайдено
    return username;
}
public async Task<string?> GetEmailAsync(int userId)
{
    _logger.LogInformation("Attempting to get email for user ID: {UserId}.",
userId);
    // Використовуйте Select та FirstOrDefaultAsync для отримання тільки
потрібного поля
    var email = await _db.Users
        .Where(u => u.UserId == userId)
        .Select(u => u.Email)
        .FirstOrDefaultAsync();

    if (email == null)
    {
        _logger.LogInformation("Email not found for user ID: {UserId}.", userId);
    }
    else
    {
        _logger.LogInformation("Email '{Email}' found for user ID: {UserId}.",
email, userId);
    }

    return email;
}

// --- Score Methods ---

// Змінюємо тип повернення на Task<EditScoreResult>
public async Task<EditScoreResult> EditScoreAsync(int userId, int score)
{
    _logger.LogInformation("Attempting to edit score for user ID: {UserId} with
value: {AttemptedScore}.", userId, score);

    // Валідація від'ємного рахунку
    if (score < 0)
    {
        _logger.LogWarning("Attempted to set negative score {AttemptedScore}
for user ID: {UserId}.", score, userId);
        // Кидаємо виняток, який контролер може обробити як BadRequest

```

```

        throw new ArgumentException("Score cannot be negative.",
nameof(score));
    }

    var scoreRecord = await _db.Scores.FindAsync(userId);

    if (scoreRecord == null)
    {
        var userExists = await _db.Users.AnyAsync(u => u.UserId == userId);
        if (!userExists)
        {
            _logger.LogWarning("Cannot create score record: User with ID
{UserId} not found in Users table.", userId);
            throw new NotFoundException(nameof(User), userId); // Користувача
не існує взагалі
        }

        // Користувач існує, але запису Score немає. Створюємо новий.
        var newScoreRecord = new Score // Припускаємо Score має властивості
UserId та ScoreValue
        {
            UserId = userId,
            ScoreValue = score,
        };

        _db.Scores.Add(newScoreRecord);
        await _db.SaveChangesAsync(); // Зберігаємо новий запис

        _logger.LogInformation("New score record created for user ID: {UserId}
with value: {AttemptedScore}.", userId, score);

        // Повертаємо результат, що вказує на створення
        return new EditScoreResult
        {
            UpdatedOrCreated = true,
            FinalScore = newScoreRecord.ScoreValue,
            StatusMessage = "Рахунок успішно збережено (створено новий
запис).",
            PreviousScore = null, // Null означає, що попереднього рахунку не
було
            AttemptedScore = score
        };
    }
    else
    {

```

```

// Запис рахунку для користувача вже існує. Порівнюємо рахунки.
if (score > scoreRecord.ScoreValue)
{
    // Новий рахунок більший за існуючий. Оновлюємо.
    var previousScoreValue = scoreRecord.ScoreValue; // Запам'ятовуємо
старий рахунок
    scoreRecord.ScoreValue = score; // Встановлюємо новий рахунок

    // EF Core відстежує зміни в scoreRecord, тому SaveChangesAsync
оновить запис в БД
    await _db.SaveChangesAsync();

    _logger.LogInformation("Score updated for user ID: {UserId} from
{PreviousScore} to {FinalScore}.", userId, previousScoreValue, score);

    // Повертаємо результат, що вказує на оновлення
return new EditScoreResult
{
    UpdatedOrCreated = true,
    FinalScore = scoreRecord.ScoreValue, // Це вже оновлене значення
    StatusMessage = "Рахунок успішно оновлено.",
    PreviousScore = previousScoreValue, // Повертаємо старий рахунок
    AttemptedScore = score
};
}
else
{
    // Новий рахунок не більший (менший або дорівнює) за існуючий.
    // Не виконуємо оновлення в базі даних.
    _logger.LogInformation($"User {userId} attempted to set score {score},
which is not higher than current score {scoreRecord.ScoreValue}. No database
update performed.");

    // Повертаємо результат, що вказує на відсутність оновлення
return new EditScoreResult
{
    UpdatedOrCreated = false, // Не було оновлення/створення
    FinalScore = scoreRecord.ScoreValue, // Повертаємо поточний (не
змінений) рахунок
    StatusMessage = $"Новий рахунок ({score}) не більший за
поточний ({scoreRecord.ScoreValue}). Оновлення не виконано.",
    PreviousScore = scoreRecord.ScoreValue,
    AttemptedScore = score
};
}
}

```

```

    }

    public async Task<int?> GetScoreAsync(int userId)
    {
        _logger.LogInformation("Attempting to get score for user ID: {UserId}.",
userId);
        // Оптимізація через Select, якщо потрібне тільки значення
        var scoreValue = await _db.Scores
            .Where(s => s.UserId == userId) // Припускаючи userId - це ключ або FK
            .Select(s => (int?)s.ScoreValue) // Проектуємо в nullable int
            .FirstOrDefaultAsync();

        if (scoreValue == null)
        {
            _logger.LogInformation("Score record not found for user ID: {UserId}.",
userId);
        }
        else
        {
            _logger.LogInformation("Score {ScoreValue} found for user ID:
{UserId}.", scoreValue.Value, userId);
        }

        // Поверне null, якщо запис Score не знайдено
        return scoreValue;
    }

    public async Task<Score[]> GetLeaderBoard()
    {
        _logger.LogInformation("Fetching leaderboard data.");
        // Вибираємо топ-10 рахунків, сортуємо за спаданням
        var leaderBord = await _db.Scores
            // .Select(s => s) // Явно вибираємо Score об'єкт (або його проекцію,
якщо потрібні не всі поля)
            .OrderByDescending(s => s.ScoreValue)
            .Take(10)
            .ToArrayAsync(); // Використовуйте ToArrayAsync або ToListAsync для
виконання запиту

        _logger.LogInformation("Fetched {Count} leaderboard entries.",
leaderBord.Length);
        return leaderBord;
    }
}

```

Додаток Б

```
using Microsoft.AspNetCore.Mvc;
using web_kursachsem4.Services;
using web_kursachsem4.Data.Models;
using System.Threading.Tasks;
using web_kursachsem4.Exceptions;
using System.Collections.Generic;
using web_kursachsem4.Web.RequestModels;
using Microsoft.Extensions.Configuration;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using Microsoft.AspNetCore.Authorization;
using Microsoft.Extensions.Logging; // Переконайтесь, що є using для логера
```

```
namespace web_kursachsem4.Web
{
    [ApiController]
    [Route("api")] // Базовий маршрут
    public class MainController : ControllerBase
    {
        private readonly ILogger<MainController> _logger;
        private readonly IMainService _mainService;
        private readonly IConfiguration _configuration;

        public MainController(ILogger<MainController> logger, IMainService
mainService, IConfiguration configuration)
        {
            _logger = logger ?? throw new ArgumentNullException(nameof(logger));
            _mainService = mainService ?? throw new
ArgumentNullException(nameof(mainService));
            _configuration = configuration ?? throw new
ArgumentNullException(nameof(configuration));
        }

        // Допоміжний метод для генерації токена
        private string GenerateJwtToken(User user)
        {
            var jwtSettings = _configuration.GetSection("JwtSettings");
            var secretKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings["SecretKey"])
            ?? throw new InvalidOperationException("JWT SecretKey is not
configured.")); // Перевірка на null
```

```

    var credentials = new SigningCredentials(secretKey,
SecurityAlgorithms.HmacSha256);

    // Перевірка на null для Issuer/Audience, якщо вони обов'язкові
    var issuer = jwtSettings["Issuer"] ?? throw new
InvalidOperationException("JWT Issuer is not configured.");
    var audience = jwtSettings["Audience"] ?? throw new
InvalidOperationException("JWT Audience is not configured.");

    var claims = new List<Claim> // Використовуємо List<Claim> для
гнучкості
    {
        // Використовуємо стандартні ClaimTypes
        new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()), // Subject
= User ID
        new Claim(ClaimTypes.Name, user.UserName), // Унікальне username
(Name)
        new Claim(ClaimTypes.Email, user.Email), // Email
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()) //
Унікальний ID токена
    };

    var token = new JwtSecurityToken(
        issuer: issuer, // Використовуємо змінну після перевірки
        audience: audience, // Використовуємо змінну після перевірки
        claims: claims,
        expires: DateTime.UtcNow.AddHours(4), // Наприклад, токен дійсний 4
години
        signingCredentials: credentials);

    return new JwtSecurityTokenHandler().WriteToken(token);
}

// Метод для отримання ID поточного користувача з токена
private int? GetCurrentUserId()
{
    // Використовуємо User.FindFirstValue та стандартні ClaimTypes
    var userIdString = User.FindFirstValue(ClaimTypes.NameIdentifier); //
Повинно відповідати ClaimTypes.NameIdentifier в GenerateJwtToken
    if (string.IsNullOrEmpty(userIdString))
    {
        _logger.LogWarning("GetCurrentUserId failed: Could not find
NameIdentifier claim in token.");
        return null;
    }
}

```

```

    }

    if (int.TryParse(userIdString, out int userId))
    {
        return userId;
    }
    else
    {
        _logger.LogError("GetCurrentUserId failed: Could not parse user ID
'{UserIdString}' from token claim into an integer.", userIdString);
        return null;
    }
}

// --- Ендпоінти для користувачів ---

// POST /api/auth/login
[HttpPost("auth/login")]
[AllowAnonymous]
[ProducesResponseType(StatusCodes.Status200OK, Type =
typeof(TokenResponse))] // Повертаємо TokenResponse
[ProducesResponseType(StatusCodes.Status400BadRequest)] //
Неправильний запит (валідація вхідних даних)
[ProducesResponseType(StatusCodes.Status401Unauthorized)] //
Неправильні логін/пароль (автентифікація не пройдена)
[ProducesResponseType(StatusCodes.Status500InternalServerError)] //
Внутрішня помилка
public async Task<ActionResult<TokenResponse>> Login([FromBody]
LoginRequest loginRequest)
{
    // Додано перевірку на null RequestModel та ModelState
    if (loginRequest == null || !ModelState.IsValid)
    {
        _logger.LogWarning("Login failed: Invalid login request received.");
        return BadRequest(ModelState); // Повертаємо деталі валідації
    }

    _logger.LogInformation("Login attempt for user: {Username}",
loginRequest.Username);

    try
    {
        var authenticatedUser = await
_mainService.AuthenticateUserAsync(loginRequest.Username,
loginRequest.Password);

```

```

        if (authenticatedUser != null)
        {
            _logger.LogInformation("User '{Username}' authenticated successfully,
generating token.", loginRequest.Username);

            // --- Генерація JWT токена ---
            var tokenString = GenerateJwtToken(authenticatedUser);
            // -----

            return Ok(new TokenResponse { Token = tokenString }); // Повертаємо
токен у TokenResponse об'єкті
        }
        else
        {
            _logger.LogWarning("Authentication failed for user '{Username}':
Invalid credentials.", loginRequest.Username);
            return Unauthorized(new { message = "Invalid username or password."
}); // 401 для невірних даних
        }
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "An error occurred during the login process for user
'{Username}'.", loginRequest.Username);
        return StatusCode(StatusCodes.Status500InternalServerError, "An
unexpected error occurred during login.");
    }
}

// POST /api/users
[HttpPost("users")]
[AllowAnonymous]
[ProducesResponseType(StatusCodes.Status201Created, Type = typeof(User))]
[ProducesResponseType(StatusCodes.Status400BadRequest)] // Валідація
вхідних даних
[ProducesResponseType(StatusCodes.Status409Conflict)] // Якщо
користувач з таким ім'ям вже існує
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<User>> AddUser([FromBody]
NewUserRequest userRequest)
{
    // Додано перевірку на null RequestModel та Model State
    if (userRequest == null || !ModelState.IsValid)

```



```

{
    _logger.LogWarning("AddUser failed: Invalid user request received.");
    return BadRequest(ModelState);
}

_logger.LogInformation("Attempting to add new user with username:
{Username}, email: {Email}", userRequest.UserName, userRequest.Email);

try
{
    User user = new User // Мапуємо RequestModel до моделі БД
    {
        UserName = userRequest.UserName,
        Email = userRequest.Email,
        Password = userRequest.Password, // Сервіс захешує його
        // Інші поля моделі User, якщо є, можливо, потрібно ініціалізувати
    };
    var addedUser = await _mainService.AddUserAsync(user);

    _logger.LogInformation("User '{Username}' created successfully with ID
{UserId}.", addedUser.UserName, addedUser.UserId);

    // Повертаємо 201 Created.
    // Припускаємо, що GetUsername(int userId) - це ендпоінт для
отримання користувача за ID.
    // Якщо такого ендпоінта немає або його назва інша, змініть nameof()
    try
    {
        return CreatedAtAction(nameof(GetUsername), new { userId =
addedUser.UserId }, addedUser);
    }
    catch (Exception ex)
    {
        // Якщо GetUsername ендпоінт не знайдено, повертаємо 200 OK
        _logger.LogError(ex, "Failed to create CreatedAtAction URL for user
{UserId}. Returning Ok.", addedUser.UserId);
        return Ok(addedUser);
    }
}
// Обробка InvalidOperationException з сервісу (наприклад, ім'я
користувача вже існує)
catch (InvalidOperationException ioex)
{
    _logger.LogWarning(ioex, "AddUser failed due to business logic:
{Message}", ioex.Message);
}
}

```

```

        return Conflict(ioex.Message); // 409 Conflict для конфлікту ресурсів
(існуюче ім'я)
        // Або BadRequest(), якщо ви вважаєте це помилкою запиту
    }
    catch (ArgumentException argEx)
    {
        // Обробка ArgumentException з сервісу (наприклад, порожні
логін/пароль)
        _logger.LogWarning(argEx, "AddUser failed due to invalid arguments:
{Message}", argEx.Message);
        return BadRequest(argEx.Message);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error adding user '{Username}'.",
userRequest.UserName);
        return StatusCode(StatusCode.Status500InternalServerError, "An
unexpected error occurred while adding the user.");
    }
}

// GET /api/me/username (Отримати ім'я поточного користувача з токена)
[HttpGet("me/username")]
[Authorize]
[ProducesResponseType(StatusCode.Status200OK, Type = typeof(string))]
[ProducesResponseType(StatusCode.Status401Unauthorized)] // Якщо немає
токена або недійсний
[ProducesResponseType(StatusCode.Status404NotFound)] // Якщо
користувача знайдено за токеном, але немає в БД (рідкісний випадок)
[ProducesResponseType(StatusCode.Status500InternalServerError)]
// Перейменовано метод, щоб уникнути конфлікту імен з GetUsername(int
userId)
public async Task<ActionResult<string>> GetMyUsername()
{
    var userId = GetCurrentUserId();
    if (userId == null)
    {
        // Цей випадок обробляється зазвичай middleware для [Authorize],
// але явна перевірка додає надійності.
        _logger.LogWarning("GetMyUsername: Unauthorized access attempt - no
valid user ID in token.");
        return Unauthorized();
    }

    _logger.LogInformation("User {UserId} requesting their own username.",
userId.Value);

```

```

try
{
    var username = await _mainService.GetUsernameAsync(userId.Value);
    if (username == null)
    {
        // Цей випадок означає, що токен валідний і містить UserID,
        // але користувача з таким ID немає в базі даних. Це може вказувати
на проблему.
        _logger.LogError("GetMyUsername: Authenticated user ID {UserId}
not found in database.", userId.Value);
        return NotFound("User data not found.");
    }
    _logger.LogInformation("Returning username '{Username}' for user ID
{UserId}.", username, userId.Value);
    return Ok(username);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error getting own username for user ID {UserId}.",
userId.Value);
    return StatusCode(StatusCode.Status500InternalServerError, "An
unexpected error occurred.");
}
}

// GET /api/me/email (Отримати email поточного користувача з токена)
[HttpGet("me/email")]
[Authorize]
[ProducesResponseType(StatusCode.Status200OK, Type = typeof(string))]
[ProducesResponseType(StatusCode.Status401Unauthorized)]
[ProducesResponseType(StatusCode.Status404NotFound)]
[ProducesResponseType(StatusCode.Status500InternalServerError)]
public async Task<ActionResult<string>> GetMyEmail() // Перейменовано
для ясності
{
    var userId = GetCurrentUserId();
    if (userId == null)
    {
        _logger.LogWarning("GetMyEmail: Unauthorized access attempt - no
valid user ID in token.");
        return Unauthorized();
    }
}

```

```

        _logger.LogInformation("User {UserId} requesting their own email.",
userId.Value);

        try
        {
            var email = await _mainService.GetEmailAsync(userId.Value);
            if (email == null)
            {
                _logger.LogError("GetMyEmail: Authenticated user ID {UserId} not
found in database for email.", userId.Value);
                return NotFound("User data not found.");
            }
            _logger.LogInformation("Returning email '{Email}' for user ID {UserId}.",
email, userId.Value);
            return Ok(email);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error getting own email for user ID {UserId}.",
userId.Value);
            return StatusCode(StatusCodes.Status500InternalServerError, "An
unexpected error occurred.");
        }
    }

    // DELETE /api/me (Видалити обліковий запис поточного користувача)
    [HttpDelete("me")]
    [Authorize]
    [ProducesResponseType(StatusCodes.Status204NoContent)] // Успішне
видалення
    [ProducesResponseType(StatusCodes.Status401Unauthorized)]
    [ProducesResponseType(StatusCodes.Status404NotFound)] // Якщо
користувача знайдено за токеном, але немає в БД
    [ProducesResponseType(StatusCodes.Status500InternalServerError)]
    public async Task<IActionResult> DeleteMyAccount() // Перейменовано для
ясності
    {
        var userId = GetCurrentUserId();
        if (userId == null)
        {
            _logger.LogWarning("DeleteMyAccount: Unauthorized access attempt -
no valid user ID in token.");
            return Unauthorized();
        }
    }

```

```

        _logger.LogInformation("User {UserId} attempting to delete their own
account.", userId.Value);

        try
        {
            await _mainService.DeleteUserAsync(userId.Value);
            _logger.LogInformation("User {UserId} deleted successfully.",
userId.Value);
            return NoContent(); // 204 NoContent - успішно, але немає тіла відповіді
        }
        // Обробка NotFoundException з сервісу, якщо користувача не знайдено
        catch (NotFoundException nfex)
        {
            _logger.LogWarning(nfex, "DeleteMyAccount failed: User with ID
{UserId} not found in database.", userId.Value);
            // Хоча токен був валідний, користувача немає. Можливо, це 404, або
401, якщо вважати сесію невалідною. 404 більш конкретно про ресурс.
            return NotFound(nfex.Message);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error deleting own account for user ID {UserId}.",
userId.Value);
            return StatusCode(StatusCodes.Status500InternalServerError, "An
unexpected error occurred while deleting the account.");
        }
    }

    // --- Ендпоінти для Score ---

    // PUT /api/me/score (Оновити рахунок поточного користувача)
    // Тепер цей ендпоінт повертає результат операції (EditScoreResult)
    [HttpPut("me/score")]
    [Authorize]
    // Оновлені ProducesResponseType: повертаємо EditScoreResult з 200 OK
    [ProducesResponseType(typeof(EditScoreResult), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)] // Наприклад,
від'ємний рахунок або ArgumentException з сервісу
    [ProducesResponseType(StatusCodes.Status401Unauthorized)] //
Забезпечується [Authorize]
    [ProducesResponseType(StatusCodes.Status404NotFound)] // Якщо
користувача знайдено за токеном, але немає в БД
    [ProducesResponseType(StatusCodes.Status500InternalServerError)]
    // Змінюємо тип повернення на ActionResult<EditScoreResult>

```

```

public async Task<ActionResult<EditScoreResult>> EditScore([FromBody]
ScoreChangeRequest scoreChangeRequest)
{
    var userId = GetCurrentUserId();
    if (userId == null)
    {
        _logger.LogWarning("EditScore: Unauthorized access attempt - no valid
user ID in token.");
        return Unauthorized();
    }

    _logger.LogInformation("User {UserId} attempting to edit their score with
value: {AttemptedScore}.", userId.Value, scoreChangeRequest?.Score);

    // Додано перевірку RequestModel на null та валідацію ModelState
    if (scoreChangeRequest == null || !ModelState.IsValid)
    {
        _logger.LogWarning("EditScore: Invalid score change request received.");
        return BadRequest(ModelState);
    }

    // Перевірка на від'ємний рахунок - може залишитись тут або тільки в
сервісі
    if (scoreChangeRequest.Score < 0)
    {
        _logger.LogWarning("EditScore: User {UserId} sent a negative score
value {AttemptedScore}.", userId.Value, scoreChangeRequest.Score);
        return BadRequest("Score cannot be negative.");
    }

    try
    {
        // Викликаємо оновлений метод сервісу, який повертає EditScoreResult
        var result = await _mainService.EditScoreAsync(userId.Value,
scoreChangeRequest.Score);

        // Логуємо результат, отриманий від сервісу
        _logger.LogInformation(
            "EditScore result for user {UserId}:
UpdatedOrCreated={UpdatedOrCreated}, FinalScore={FinalScore},
Message='{Message}'",
            userId.Value, result.UpdatedOrCreated, result.FinalScore,
result.StatusMessage);
    }
}

```

```

// Повертаємо статус 200 OK разом з об'єктом результату.
// Об'єкт EditScoreResult пояснить клієнту, чи було оновлення.
return Ok(result);

}
// Обробка ArgumentException з сервісу (наприклад, якщо score < 0, хоча
ми перевіряємо вище)
catch (ArgumentException argEx) when (argEx.ParamName == "score")
{
    _logger.LogWarning(argEx, "EditScore failed due to invalid score value
from service: {Message}", argEx.Message);
    return BadRequest(argEx.Message);
}
// Обробка NotFoundException, якщо користувача не знайдено в сервісі
(малоймовірно при Authorize)
catch (NotFoundException nfex)
{
    _logger.LogWarning(nfex, "EditScore failed: User with ID {UserId} not
found in database according to service.", userId.Value);
    return NotFound(nfex.Message); // 404, якщо сервіс каже, що
користувача немає
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error editing score for user ID {UserId}.",
userId.Value);
    return StatusCode(StatusCodes.Status500InternalServerError, "An
unexpected error occurred while editing the score.");
}
}

// GET /api/me/score (Отримати рахунок поточного користувача)
[HttpGet("me/score")]
[Authorize]
[ProducesResponseType(StatusCodes.Status200OK, Type = typeof(int))]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)] // Якщо
користувача або його рахунку немає
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<int>> GetMyScore() // Перейменовано для
ясності
{
    var userId = GetCurrentUserId();
    if (userId == null)
    {

```

```

        _logger.LogWarning("GetMyScore: Unauthorized access attempt - no valid
user ID in token.");
        return Unauthorized();
    }

```

```

        _logger.LogInformation("User {UserId} requesting their own score.",
userId.Value);

```

```

    try
    {
        // Викликаємо сервіс з ID з токена
        var score = await _mainService.GetScoreAsync(userId.Value);
        if (score == null)
        {
            // Якщо GetScoreAsync повернув null (запис Score не знайдено для
цього UserID)
            _logger.LogInformation("Score data not found for user ID {UserId}.",
userId.Value);
            return NotFound("Score data not found.");
        }
        _logger.LogInformation("Returning score {ScoreValue} for user ID
{UserId}.", score.Value, userId.Value);
        return Ok(score.Value); // Повертаємо int
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error getting own score for user ID {UserId}.",
userId.Value);
        return StatusCode(StatusCodes.Status500InternalServerError, "An
unexpected error occurred while retrieving the score.");
    }
}
// GET /api/leaderboard (Отримати топ-10 рахунків)
[HttpGet("leaderboard")]
[AllowAnonymous] // Дозволяємо всім дивитись таблицю лідерів
[ProducesResponseType(StatusCodes.Status200OK, Type = typeof(Score[]))] //
Повертаємо масив об'єктів Score
// ProducesResponseType 404 не потрібен, бо сервіс поверне порожній
масив, а не null/NotFoundException
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult<Score[]>> GetLeaderboard()
{
    _logger.LogInformation("Request received for leaderboard.");

    try

```



```

{
    // Сервіс GetLeaderBoard тепер повертає Score[], а не null
    var leaderboard = await _mainService.GetLeaderBoard();

    // Перевірка на null більше не потрібна тут, бо ToArrayAsync повертає
    // порожній масив, якщо немає даних.
    // if (leaderboard == null) // Це було б некоректно з ToArrayAsync()
    // {
    //     _logger.LogInformation($"No data for leaderboard");
    //     return NotFound("Leaderboard data not found."); // Можна повернути
    //     Ок з порожнім масивом замість NotFound
    // }

    _logger.LogInformation("Returning {Count} leaderboard entries.",
    leaderboard.Length);
    return Ok(leaderboard); // Повертаємо 200 ОК з масивом (можливо,
    порожнім)
}
catch (Exception ex)
{
    _logger.LogError(ex, "Exception Caught while fetching leaderboard.");
    return StatusCode(StatusCode.Status500InternalServerError, "An
    unexpected error occurred while retrieving the leaderboard.");
}
}

// --- Захищений ендпоінт для отримання даних ІНШОГО користувача ---

// GET /api/users/{userId:int}/username (Отримати ім'я будь-якого
користувача за ID)
// Використовується в CreatedAtAction
[HttpGet("users/{userId:int}/username")] // Маршрут з параметром ID
[Authorize] // Вимагає лише автентифікації, щоб дивитись імена інших
[ProducesResponseType(StatusCode.Status200OK, Type = typeof(string))]
[ProducesResponseType(StatusCode.Status401Unauthorized)]
[ProducesResponseType(StatusCode.Status404NotFound)]
[ProducesResponseType(StatusCode.Status500InternalServerError)]
// Залишаємо оригінальне ім'я методу GetUsername, оскільки воно
використовується в CreatedAtAction
public async Task<ActionResult<string>> GetUsername(int userId) // Приймає
userId з URL
{
    var currentUserId = GetCurrentUser(); // Отримуємо ID того, хто запитує,
    для логування

```

```

        _logger.LogInformation("User {CurrentUserId} requesting username for user
ID {TargetUserId}", currentUserId?.ToString() ?? "Unknown", userId);

        try
        {
            // Викликаємо сервіс з ID з URL
            var username = await _mainService.GetUsernameAsync(userId);
            if (username == null)
            {
                _logger.LogInformation("Username not found for target user ID
{TargetUserId}.", userId);
                return NotFound($"User with ID {userId} not found.");
            }
            _logger.LogInformation("Returning username '{Username}' for target user
ID {TargetUserId}.", username, userId);
            return Ok(username);
        }
        // Немає специфічних винятків для обробки від GetUsernameAsync (він
повертає null на 404)
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error getting username for user ID {UserId}.",
userId);
            return StatusCode(StatusCodes.Status500InternalServerError, "An
unexpected error occurred.");
        }
    }

    // Можна видалити ваш тестовий ендпоінт або залишити для швидкої
перевірки
    [HttpGet("/api/test")]
    [AllowAnonymous]
    [ApiExplorerSettings(IgnoreApi = true)] // Не показувати в Swagger UI
    public ActionResult GetTest()
    {
        _logger.LogInformation("Test endpoint called.");
        return Ok("API is running ok!");
    }
}

```